


DEPARTMENT OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE
INDIAN INSTITUTE OF TECHNOLOGY MADRAS
CHENNAI – 600036

# LLM-Guided Code Synthesis for Boundary Element Methods

*A Thesis*

*Submitted by*

**PRANAV**

*For the award of the degree*

*Of*

**INTERDISCIPLINARY DUAL DEGREE**

May 2026




DEPARTMENT OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE
INDIAN INSTITUTE OF TECHNOLOGY MADRAS
CHENNAI – 600036

# LLM-Guided Code Synthesis for Boundary Element Methods

*A Thesis*

*Submitted by*

**PRANAV**

*For the award of the degree*

*Of*

**INTERDISCIPLINARY DUAL DEGREE**

May 2026

© 2026 Indian Institute of Technology Madras

# THESIS CERTIFICATE

This is to undertake that the Thesis titled **LLM-GUIDED CODE SYNTHESIS FOR BOUNDARY ELEMENT METHODS**, submitted by me to the Indian Institute of Technology Madras, for the award of **INTERDISCIPLINARY DUAL DEGREE**, is a bona fide record of the research work done by me under the supervision of **Dr. Sivaram Ambikasaran and Dr. Sri Vallabha Deevi**. The contents of this Thesis, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

**Chennai 600036** **Pranav**

**Date: May 2026**

**Dr. Sivaram Ambikasaran**
Associate Professor
Department of Data Science and Artificial Intelligence
IIT Madras

**Dr. Sri Vallabha Deevi**
Director, Data Science, Tiger Analytics
Adjunct Faculty
Department of Data Science and Artificial Intelligence
IIT Madras

© 2026 Indian Institute of Technology Madras

# ACKNOWLEDGEMENTS

I express my deepest gratitude to my guides, Dr. Sivaram Ambikasaran and Dr. Sri Vallabha Deevi, for their invaluable guidance, insightful discussions, and continuous support throughout this thesis. Their combined expertise and constructive feedback were instrumental in shaping this exploration of Large Language Model-guided code generation for Boundary Element Methods.

I extend my sincere thanks to the Department of Data Science at the Indian Institute of Technology Madras for providing an exceptional academic environment to carry out this work. I would also like to thank my thesis evaluation committee for their time and valuable feedback.

Finally, I am sincerely grateful to my parents and family for their constant support, patience, and encouragement. I would also like to thank my friends for keeping me grounded and providing the necessary balance outside of my academic responsibilities.

# ABSTRACT

The Boundary Element Method (BEM) is a highly efficient numerical technique for solving boundary value problems, but its practical adoption is often bottlenecked by the complexity of its mathematical implementation, including singular integrations and dense matrix operations. While Large Language Models (LLMs) have demonstrated significant proficiency in general-purpose software engineering, their capacity to autonomously synthesize niche, highly mathematical scientific solvers remains under-explored.

This thesis presents an empirical evaluation of LLM-guided code generation for BEM solvers applied to standard 2D and 3D geometries. Three state-of-the-art foundation models—Gemini Pro, ChatGPT (Thinking), and Claude Sonnet — were systematically tested. To ensure numerical stability and physical accuracy, the models were explicitly constrained to well-conditioned mathematical frameworks.

The generated solvers were benchmarked for generative efficiency, spatial convergence, and computational scalability. Specifically, this study explored the zero-shot mathematical reasoning capabilities of these foundation models in handling core algorithmic logic and boundary integrations for known 2D and 3D shapes, presenting a comparative analysis of their performance and computational behavior.

# CONTENTS


Page

ABSTRACT iii

LIST OF TABLES vii

LIST OF FIGURES xiii

ABBREVIATIONS xv

CHAPTER 1 INTRODUCTION AND MOTIVATION 1

1.1 The Bottleneck in Computational Engineering . . . . . 1
1.2 Introduction to BEM . . . . . 2
1.3 The LLM Paradigm in Scientific Computing . . . . . 4
1.4 Research Objective . . . . . 5

CHAPTER 2 METHODOLOGY AND PROMPTING FRAMEWORK 7

2.1 Taxonomy of BEM Formulations . . . . . 7
2.2 Theoretically Driven Method Selection . . . . . 9
2.3 The Prompting Pipeline and Iterative Feedback . . . . . 11
2.4 Evaluation Metrics . . . . . 13

CHAPTER 3 LLM CODE SYNTHESIS FOR 2D DOMAINS 15

3.1 Introduction to 2D Testing . . . . . 15
3.2 Part I: Basic 2D Geometries . . . . . 15
3.2.1 The Unit Circle . . . . . 15
3.2.2 Annulus . . . . . 22
3.2.3 Rectangle . . . . . 29
3.2.4 Triangle . . . . . 31
3.2.5 Ellipse . . . . . 37
3.3 Part II: Complex 2D Geometries . . . . . 42
3.3.1 Smooth Star-Shaped Domain . . . . . 42
3.3.2 M-Shaped Domain . . . . . 48
3.3.3 NACA 4318 Airfoil Domain . . . . . 54
3.4 Cross-Domain Knowledge Transfer . . . . . 58

CHAPTER 4 LLM CODE SYNTHESIS FOR 3D DOMAINS 65

4.1 Introduction to 3D Testing . . . . . 65
4.2 Numerical results for 3D domains . . . . . 65
4.2.1 Sphere . . . . . 65
4.2.2 Cube . . . . . 68
4.2.3 Skewed parallelepiped . . . . . 72

4.2.4 Perturbed Sphere . . . . . 77

**CHAPTER 5 CONCLUSIONS AND FUTURE WORK 83**

5.1 Summary of Findings . . . . . 83
5.2 Concluding Remarks . . . . . 84
5.3 Directions for Future Work . . . . . 84

**REFERENCES 87**

**CURRICULUM VITAE 89**

# LIST OF TABLES


| Table | Caption | Page |
|---|---|---|
| 3.1 | Mesh refinement study of the Gemini Pro Indirect DLP solver using constant collocation on the unit circle. | 16 |
| 3.2 | Mesh refinement study of the ChatGPT Indirect DLP solver using constant collocation on the unit circle. | 17 |
| 3.3 | Mesh refinement study of the Claude Sonnet Indirect DLP solver using constant collocation on the unit circle. | 17 |
| 3.4 | Mesh refinement study of the Gemini Pro Indirect DLP solver using linear collocation on the unit circle. | 18 |
| 3.5 | Mesh refinement study of the ChatGPT Indirect DLP solver using linear collocation on the unit circle. | 18 |
| 3.6 | Mesh refinement study of the Claude Sonnet Indirect DLP solver using linear collocation on the unit circle. | 18 |
| 3.7 | Mesh refinement study of the Gemini Pro Indirect DLP solver using linear galerkin on the unit circle. | 19 |
| 3.8 | Mesh refinement study of the ChatGPT Indirect DLP solver using linear galerkin on the unit circle. | 20 |
| 3.9 | Mesh refinement study of the Claude Sonnet Indirect DLP solver using linear galerkin on the unit circle. | 20 |
| 3.10 | Mesh refinement study of the Gemini Pro Spectral BEM solver on the unit circle. | 21 |
| 3.11 | Mesh refinement study of the ChatGPT Spectral BEM solver on the unit circle. | 21 |
| 3.12 | Mesh refinement study of the Claude Sonnet Spectral BEM solver on the unit circle. | 21 |
| 3.13 | Mesh refinement study of the Gemini Pro Direct Constant Collocation BEM solver on the annular domain. | 24 |
| 3.14 | Mesh refinement study of the ChatGPT Direct Constant Collocation BEM solver on the annular domain. | 24 |
| 3.15 | Mesh refinement study of the Claude Sonnet Direct Constant Collocation BEM solver on the annular domain. | 24 |
| 3.16 | Mesh refinement study of the Gemini Pro Direct Linear Collocation BEM solver on the annular domain. | 25 |
| 3.17 | Mesh refinement study of the ChatGPT Direct Linear Collocation BEM solver on the annular domain. | 25 |
| 3.18 | Mesh refinement study of the Claude Sonnet Direct Linear Collocation BEM solver on the annular domain. | 26 |
| 3.19 | Mesh refinement study of the Gemini Pro Direct Linear Galerkin BEM solver on the annular domain. | 27 |
| 3.20 | Mesh refinement study of the Claude Sonnet Direct Linear Galerkin BEM solver on the annular domain. | 27 |
| 3.21 | Performance for indirect formulations with mixed boundary conditions. | 28 |

3.22 Metrics at the finest mesh ($M = 2560$) . . . . . 30
3.23 Metrics at the finest mesh ($M = 2560$) . . . . . 30
3.24 Metrics at the finest mesh ($M = 640$) . . . . . 31
3.25 Mesh refinement study of the Gemini Pro Indirect Combined Potential Constant Collocation BEM solver on the triangular domain. . . . . 32
3.26 Mesh refinement study of the ChatGPT Indirect Combined Potential Constant Collocation BEM solver on the triangular domain. . . . . 33
3.27 Mesh refinement study of the Claude Sonnet Indirect Combined Potential Constant Collocation BEM solver on the triangular domain. . . . . 33
3.28 Mesh refinement study of the Gemini Pro Indirect DLP Linear Collocation BEM solver on the triangular domain. . . . . 34
3.29 Mesh refinement study of the ChatGPT Indirect DLP Linear Collocation BEM solver on the triangular domain. . . . . 34
3.30 Mesh refinement study of the Claude Sonnet Indirect DLP Linear Collocation BEM solver on the triangular domain. . . . . 34
3.31 Mesh refinement study of the Gemini Pro Indirect Combined Potential Linear Collocation BEM solver on the triangular domain. . . . . 35
3.32 Mesh refinement study of the ChatGPT Indirect Combined Potential Linear Collocation BEM solver on the triangular domain. . . . . 35
3.33 Mesh refinement study of the Claude Sonnet Indirect Combined Potential Linear Collocation BEM solver on the triangular domain. . . . . 36
3.34 Mesh refinement study of the Gemini Pro Indirect DLP Constant Collocation BEM solver on the elliptical domain. . . . . 38
3.35 Mesh refinement study of the ChatGPT Indirect DLP Constant Collocation BEM solver on the elliptical domain. . . . . 38
3.36 Mesh refinement study of the Claude Indirect DLP Constant Collocation BEM solver on the elliptical domain. . . . . 38
3.37 Mesh refinement study of the Gemini Pro Indirect DLP Linear Collocation BEM solver on the Elliptical domain. . . . . 39
3.38 Mesh refinement study of the ChatGPT Indirect DLP Linear Collocation BEM solver on the Elliptical domain. . . . . 39
3.39 Mesh refinement study of the Claude Indirect DLP Linear Collocation BEM solver on the Elliptical domain. . . . . 40
3.40 Mesh refinement study of the Gemini Pro Indirect DLP Linear Galerkin BEM solver on the Elliptical domain. . . . . 41
3.41 Mesh refinement study of the ChatGPT Indirect DLP Linear Galerkin BEM solver on the Elliptical domain. . . . . 41
3.42 Mesh refinement study of the Claude Indirect DLP Linear Galerkin BEM solver on the Elliptical domain. . . . . 41
3.43 Mesh refinement study of the Gemini Pro Indirect DLP Constant Collocation BEM solver on the Star-shaped domain. . . . . 44
3.44 Mesh refinement study of the ChatGPT Indirect DLP Constant Collocation BEM solver on the Star-shaped domain. . . . . 44
3.45 Mesh refinement study of the Claude Indirect DLP Constant Collocation BEM solver on the Star-shaped domain. . . . . 44

3.46 Mesh refinement study of the Gemini Pro Indirect DLP Linear Collocation BEM solver on the Star-shaped domain. . . . . 45
3.47 Mesh refinement study of the ChatGPT Indirect DLP Linear Collocation BEM solver on the Star-shaped domain. . . . . 45
3.48 Mesh refinement study of the Claude Indirect DLP Linear Collocation BEM solver on the Star-shaped domain. . . . . 46
3.49 Mesh refinement study of the Gemini Pro Indirect DLP Linear Galerkin BEM solver on the Star-shaped domain. . . . . 47
3.50 Mesh refinement study of the ChatGPT Indirect DLP Linear Galerkin BEM solver on the Star-shaped domain. . . . . 47
3.51 Mesh refinement study of the Claude Indirect DLP Linear Galerkin BEM solver on the Star-shaped domain. . . . . 47
3.52 Mesh refinement study of the Gemini Pro Indirect DLP Constant Collocation BEM solver on the M-shaped domain. . . . . 50
3.53 Mesh refinement study of the ChatGPT Indirect DLP Constant Collocation BEM solver on the M-shaped domain. . . . . 50
3.54 Mesh refinement study of the Claude Indirect DLP Constant Collocation BEM solver on the M-shaped domain. . . . . 50
3.55 Mesh refinement study of the Gemini Pro Indirect DLP Linear Collocation BEM solver on the M-shaped domain. . . . . 51
3.56 Mesh refinement study of the ChatGPT Indirect DLP Linear Collocation BEM solver on the M-shaped domain. . . . . 51
3.57 Mesh refinement study of the Claude Indirect DLP Linear Collocation BEM solver on the M-shaped domain. . . . . 52
3.58 Mesh refinement study of the Gemini Pro Indirect DLP Linear Galerkin BEM solver on the M-shaped domain. . . . . 52
3.59 Mesh refinement study of the ChatGPT Indirect DLP Linear Galerkin BEM solver on the M-shaped domain. . . . . 53
3.60 Mesh refinement study of the Gemini Pro Direct Constant Collocation BEM solver on the NACA 4318 domain. . . . . 55
3.61 Mesh refinement study of the ChatGPT Direct Constant Collocation BEM solver on the NACA 4318 domain. . . . . 55
3.62 Initial mesh refinement study of the Claude Direct Constant Collocation BEM solver on the NACA 4318 domain (Anti-convergent). . . . . 55
3.63 Mesh refinement study of the Gemini Pro Direct Linear Collocation BEM solver on the NACA 4318 domain. . . . . 56
3.64 Mesh refinement study of the ChatGPT Direct Linear Collocation BEM solver on the NACA 4318 domain. . . . . 56
3.65 Mesh refinement study of the Claude Direct Linear Collocation BEM solver on the NACA 4318 domain. . . . . 56
3.66 Mesh refinement study of the Gemini Pro Direct Linear Galerkin BEM solver on the NACA 4318 domain. . . . . 57
3.67 Mesh refinement study of the ChatGPT Direct Linear Galerkin BEM solver on the NACA 4318 domain. . . . . 57
3.68 Mesh refinement study of the Claude Direct Linear Galerkin BEM solver on the NACA 4318 domain. . . . . 58

3.69 Mesh refinement study of the Gemini Pro Direct Constant Collocation BEM solver on the Rectangular domain (One-Shot). . . . . 60
3.70 Mesh refinement study of the ChatGPT Direct Constant Collocation BEM solver on the Rectangular domain (One-Shot). . . . . 61
3.71 Mesh refinement study of the Claude Direct Constant Collocation BEM solver on the Rectangular domain (One-Shot). . . . . 61
3.72 Mesh refinement study of the Gemini Pro Direct Linear Collocation BEM solver on the Rectangular domain (One-Shot). . . . . 62
3.73 Mesh refinement study of the ChatGPT Direct Linear Collocation BEM solver on the Rectangular domain (One-Shot). . . . . 62
3.74 Mesh refinement study of the Claude Direct Linear Collocation BEM solver on the Rectangular domain (One-Shot). . . . . 62
3.75 Mesh refinement study of the Gemini Pro Direct Linear Galerkin BEM solver on the Rectangular domain (One-Shot). . . . . 63
3.76 Mesh refinement study of the ChatGPT Direct Linear Galerkin BEM solver on the Rectangular domain (One-Shot). . . . . 63
3.77 Mesh refinement study of the Claude Direct Linear Galerkin BEM solver on the Rectangular domain (One-Shot). . . . . 63

4.1 Mesh refinement study of the Gemini Pro Indirect DLP Constant Collocation BEM solver on the Sphere domain. . . . . 66
4.2 Mesh refinement study of the ChatGPT Indirect DLP Constant Collocation BEM solver on the Sphere domain. . . . . 66
4.3 Mesh refinement study of the Gemini Pro Indirect Collocation Linear BEM solver on the Sphere domain. . . . . 67
4.4 Mesh refinement study of the ChatGPT Indirect Collocation Linear BEM solver on the Sphere domain. . . . . 67
4.5 Mesh refinement study of the Gemini Pro Indirect DLP Galerkin Linear BEM solver on the Sphere domain. . . . . 67
4.6 Mesh refinement study of the ChatGPT Indirect DLP Galerkin Linear BEM solver on the Sphere domain. . . . . 68
4.7 Mesh refinement study of the Gemini Pro Direct Collocation Constant BEM solver on the Cube domain. . . . . 69
4.8 Mesh refinement study of the ChatGPT Direct Collocation Constant BEM solver on the Cube domain. . . . . 70
4.9 Mesh refinement study of the Gemini Direct Linear Collocation BEM solver on the Cube domain. . . . . 70
4.10 Mesh refinement study of the ChatGPT Direct Linear Collocation BEM solver on the Cube domain. . . . . 71
4.11 Mesh refinement study of the Gemini Pro Direct Linear Galerkin BEM solver on the 3D Cube domain (Refined). . . . . 71
4.12 Mesh refinement study of the ChatGPT Direct Linear Galerkin BEM solver on the 3D Cube domain (Failed Iteration). . . . . 72
4.13 Mesh refinement study of the Gemini Pro Indirect Constant Collocation BEM solver on the Skewed Parallelepiped domain. . . . . 74
4.14 Mesh refinement study of the ChatGPT Direct Constant Collocation BEM solver on the Skewed Parallelepiped domain. . . . . 74

4.15 Mesh refinement study of the Gemini Pro Indirect DLP Linear BEM solver on the Skewed Parallelepiped domain. . . . . 75
4.16 Mesh refinement study of the ChatGPT Indirect DLP Linear BEM solver on the Skewed Parallelepiped domain. . . . . 75
4.17 Mesh refinement study of the Gemini Pro Indirect DLP Galerkin Linear BEM solver on the Skewed Parallelepiped domain. . . . . 76
4.18 Mesh refinement study of the ChatGPT Indirect DLP Galerkin Linear BEM solver on the Skewed Parallelepiped domain. . . . . 76
4.19 Mesh refinement study of the Gemini Pro Direct Collocation Constant BEM solver on the Perturbed Sphere domain. . . . . 79
4.20 Mesh refinement study of the ChatGPT Direct Collocation Constant BEM solver on the Perturbed Sphere domain. . . . . 79
4.21 Mesh refinement study of the Gemini Pro Direct Collocation Linear BEM solver on the Perturbed Sphere domain. . . . . 80
4.22 Mesh refinement study of the ChatGPT Direct Collocation Linear BEM solver on the Perturbed Sphere domain. . . . . 80
4.23 Mesh refinement study of the Gemini Pro Direct Galerkin Linear BEM solver on the Perturbed Sphere domain. . . . . 81
4.24 Mesh refinement study of the ChatGPT Direct Galerkin Linear BEM solver on the Perturbed Sphere domain. . . . . 81

# LIST OF FIGURES


| Figure | Caption | Page |
|---|---|---|
| 3.1 | $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for Circle | 22 |
| 3.2 | $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for Annulus | 28 |
| 3.3 | $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for Triangle | 36 |
| 3.4 | $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for Ellipse | 42 |
| 3.5 | Boundary of the smooth star-shaped domain. | 42 |
| 3.6 | $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for Star-shaped domain | 48 |
| 3.7 | Boundary of the non-convex M-shaped domain. | 49 |
| 3.8 | $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for M-shaped domain | 53 |
| 3.9 | NACA 4318 airfoil domain with mixed Dirichlet-Neumann boundary conditions. | 54 |
| 3.10 | $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for NACA 4318 Airfoil | 58 |
| 3.11 | One-shot prompt template utilized for cross-domain knowledge transfer. | 59 |
| 3.12 | $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for Rectangle under one-shot prompting | 64 |
| 4.1 | $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for the sphere domain | 68 |
| 4.2 | $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for the cube | 72 |
| 4.3 | Skewed Parallelepiped domain | 73 |
| 4.4 | $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for the skewed parallelepiped domain | 77 |
| 4.5 | Perturbed Sphere domain | 78 |
| 4.6 | $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for the perturbed sphere domain | 82 |

# ABBREVIATIONS

**BEM** Boundary Element Methods.

**DLP** Double Layer Potential.

**LLM** Large Language Model.

**SLP** Single Layer Potential.

# CHAPTER 1

# INTRODUCTION AND MOTIVATION

## 1.1 THE BOTTLENECK IN COMPUTATIONAL ENGINEERING

The Boundary Element Method (BEM) [1] is a numerical technique in computational engineering, standing alongside methods like Finite Element Analysis (FEA) and volume-based solvers. Its primary advantage is dimensionality reduction. By formulating governing equations—such as the Laplace equation—strictly on the boundary of a domain, BEM reduces a 3D volume problem to a 2D surface problem, or a 2D area to a 1D contour. This mathematical formulation not only decreases the size of the required computational mesh but also makes BEM suited for modeling infinite domains where traditional methods struggle to define outer boundaries.

However, a fundamental distinction between FEA and BEM lies in their handling of geometric complexity. FEA abstracts geometry entirely into the meshing process, allowing a general solver architecture to process arbitrary domains. In contrast, BEM formulations are sensitive to geometric nuances. Features such as sharp corners, thin domains, or multiply-connected topologies disrupt standard boundary integral evaluations, necessitating specialized algorithmic logic and tailored singularity handling.

Consequently, despite its theoretical advantages, translating BEM theory into a working computational solver is difficult and time-consuming. A researcher building a custom BEM solver from scratch must manually write algorithms to handle tasks like singular integration—managing calculations where distances between points approach zero—and the assembly of dense matrices. Even a minor human error, such as a misplaced negative sign, a flipped normal vector, or an improperly scaled theoretical constant, will cause the simulation to fail or output incorrect results.

Because of this geometric sensitivity and mathematical rigidity, the primary bottleneck in computational engineering research is often human development time rather than processing power. The difficulty of translating boundary integral formulations into code demands a new paradigm for solver synthesis. There is a need for automated workflows capable of translating theoretical physics into computational pipelines, mitigating the manual debugging process without sacrificing mathematical integrity.

To address this challenge, this study investigates the capability of Large Language Models (LLMs) to automatically generate and accelerate BEM solvers. Using the Laplace equation across various geometries as a test bed, we prompt foundation models—such as GPT, Claude, and Gemini—to synthesize custom BEM code. By iterating on these prompts and evaluating the generated solvers against known analytical solutions, this research explores the feasibility of using AI-assisted coding to alleviate the traditional bottlenecks, documenting the successes and failure modes encountered while attempting to automate solver generation.

## 1.2 INTRODUCTION TO BEM

The Boundary Element Method (BEM) is a computational technique utilized to solve linear partial differential equations (PDEs) which have been mathematically reformulated as integral equations. For the scope of this study, the focus is restricted to the Laplace equation, a PDE governing steady-state, divergence-free phenomena such as heat conduction, ideal fluid flow, and electrostatics. The Laplace equation for an unknown potential $u$ within a given domain $\Omega$ is defined as:

$$\nabla^2 u(\mathbf{x}) = 0, \quad \mathbf{x} \in \Omega \tag{1.1}$$

The mathematical formulation of BEM relies on Green's second identity and the concept of a fundamental solution, or free-space Green's function. The fundamental solution, denoted as $u^*$, represents the potential field generated by a concentrated point source

in an infinite domain. For the 2D Laplace equation, the fundamental solution and its normal derivative (flux, $q^*$) are defined as:

$$u^*(\mathbf{x}, \mathbf{y}) = -\frac{1}{2\pi} \ln(r), \quad q^*(\mathbf{x}, \mathbf{y}) = \frac{\partial u^*}{\partial n} \tag{1.2}$$

where $r = |\mathbf{x} - \mathbf{y}|$ is the Euclidean distance between the source point $\mathbf{x}$ and the field point $\mathbf{y}$, and $n$ is the outward unit normal vector.

By applying these principles, the governing volumetric PDE is transformed into a Boundary Integral Equation (BIE). This transformation dictates that the solution at any point can be determined by the potential ($u$) and its flux ($q$) defined along the domain boundary $\Gamma$:

$$c(\mathbf{x})u(\mathbf{x}) + \int_\Gamma q^*(\mathbf{x}, \mathbf{y})u(\mathbf{y})d\Gamma = \int_\Gamma u^*(\mathbf{x}, \mathbf{y})q(\mathbf{y})d\Gamma \tag{1.3}$$

Here, $c(\mathbf{x})$ is the geometric jump term (or solid angle coefficient), which equals 1 for an interior point and 0.5 for a point residing on a smooth boundary. Properly identifying and implementing this jump term is a theoretical requirement when coding BEM solvers.

To solve this BIE computationally, the continuous boundary $\Gamma$ is discretized into finite segments known as elements—such as constant, linear, or quadratic elements. The integral equation is then evaluated across these discrete elements, generating a system of linear algebraic equations. This system is expressed in matrix form as:

$$[H]\{u\} = [G]\{q\} \tag{1.4}$$

where $[H]$ and $[G]$ are the influence matrices assembled from the singular and non-singular integration of $q^*$ and $u^*$, respectively.

By applying known boundary conditions—whether Dirichlet (known potential), Neumann (known flux), or Mixed—this system is rearranged to group all unknown variables into a single vector $x$, taking the standard form $Ax = b$. Once the linear system is solved and the state of the boundary is resolved, the field variables at any internal point within the

domain $\Omega$ can be calculated as a post-processing step, bypassing the need for an internal volumetric mesh.

### 1.3 THE LLM PARADIGM IN SCIENTIFIC COMPUTING

The advent of Large Language Models (LLMs) has influenced automated code generation. Built upon transformer architectures [2], these probabilistic engines excel at linguistic approximation and pattern recognition. In standard software engineering, tasks are often structurally flexible, allowing multiple syntactical approaches to execute a functional script. Consequently, LLMs have achieved success in accelerating general-purpose programming.

However, transitioning these models into the domain of mathematical physics exposes limitations. Computational mechanics is intolerant of approximation. When an LLM is prompted to generate a custom BEM solver, it must adhere to theoretical constraints rather than relying on code that superficially resembles physics. Left to their probabilistic tendencies, LLMs exhibit three failure modes in BEM generation:

- **Mishandling Singularities:** Models often fail to recognize boundary singularities, applying standard numerical integration—such as basic Gaussian quadrature—where analytical extraction of logarithmic or reciprocal kernels is required.
- **Theoretical Omissions:** LLMs neglect mathematical necessities, such as the geometric jump term $c(\mathbf{x})$ defined in the previous section, treating it as an arbitrary constant to be dropped rather than a required property of the boundary integral.
- **Formulation Hallucination:** Without guidance, models blend incompatible mathematical frameworks—such as mixing Collocation matrix assembly logic with Galerkin double-integration requirements—yielding code that compiles but produces non-physical data.

Therefore, prompting an LLM to "write a BEM solver" is an inadequate methodology for scientific computing. To utilize LLMs in the domain of mathematical physics, they

must be constrained. This thesis addresses this gap by treating the LLM not as an autonomous code generator, but as a probabilistic engine that requires guidance. By enforcing physics-informed prompting, utilizing iterative mathematical feedback loops to correct theoretical violations, and employing few-shot knowledge transfer to map functional logic across geometries, this research demonstrates how to bridge the gap between linguistic approximation and deterministic computational physics.

## 1.4 RESEARCH OBJECTIVE

The primary objective of this thesis is to evaluate and improve the capability of Large Language Models (LLMs) to generate functional Boundary Element Method (BEM) solvers. Rather than treating LLMs as autonomous code generators, this study adopts an empirical, iterative approach to code synthesis based on execution and feedback.

Specifically, this research aims to formalize a workflow for AI-assisted BEM development through the following steps:

- **Standardized Prompting:** Developing and utilizing a structured prompt template that defines the BEM problem—including the target geometry, boundary conditions, and mathematical formulation—to generate an initial baseline solver.
- **Execution and Evaluation:** Executing the initial code generated by foundation models—such as GPT, Claude, and Gemini—to evaluate its performance, syntax, and mathematical accuracy against known analytical solutions.
- **Iterative Feedback Loop:** Supplying the execution results—including error logs, non-physical outputs, or divergent behavior—back to the LLM. If the initial solver fails to converge, the model is sequentially prompted to debug and adjust the underlying mathematical formulation until a functional solver is achieved.

By applying this prompting and feedback loop across various 2D and 3D geometries, this research documents the interactions and iterative steps required to guide an LLM in generating computationally accurate physics code.

# CHAPTER 2

# METHODOLOGY AND PROMPTING FRAMEWORK

## 2.1 TAXONOMY OF BEM FORMULATIONS

To systematically evaluate the code-generation capabilities of Large Language Models (LLMs), it is first necessary to establish a rigorous taxonomy of Boundary Element Method (BEM) formulations. The mathematical architecture of a BEM solver is not singular; rather, it is defined by a combination of theoretical and numerical choices. Different formulations impose distinct computational requirements, singular integration techniques, and matrix assembly logic.

For the purposes of this study, the tested LLM-generated solvers are categorized across three methodological axes:

**1. Integral Formulation: Direct vs. Indirect**

- **Direct Formulation:** This approach [3] derives the boundary integral equation using Green's second identity, formulating the system directly in terms of the physical variables: the boundary potential $u$ and the normal flux $q$. For a source point $\mathbf{x}$ and field point $\mathbf{y}$, it is expressed as:

$$c(\mathbf{x})u(\mathbf{x}) + \int_\Gamma q^*(\mathbf{x},\mathbf{y})u(\mathbf{y})d\Gamma = \int_\Gamma u^*(\mathbf{x},\mathbf{y})q(\mathbf{y})d\Gamma \tag{2.1}$$

- **Indirect Formulation:** Proposed by Kupradze [4], this approach states that instead of physical variables, the potential field is modeled using fictitious continuous source densities distributed over the boundary. These are typically expressed as a Single-Layer Potential with density $\sigma(\mathbf{y})$, or a Double-Layer Potential with density $\mu(\mathbf{y})$:

$$u(\mathbf{x}) = \int_\Gamma u^*(\mathbf{x},\mathbf{y})\sigma(\mathbf{y})d\Gamma \quad \text{or} \quad u(\mathbf{x}) = \int_\Gamma q^*(\mathbf{x},\mathbf{y})\mu(\mathbf{y})d\Gamma \tag{2.2}$$

**2. Method of Weighted Residuals: Collocation vs. Galerkin**

Once the boundary integral equation is established, it inherently produces a numerical residual $R(\mathbf{x})$ when approximated. The method chosen to minimize this residual alters the solver's architecture:

- **Collocation Method:** The residual is forced to zero exactly at a set of $N$ discrete nodes—or collocation points—$\mathbf{x}_i$, effectively using the Dirac delta function $\delta(\mathbf{x}-\mathbf{x}_i)$ as the weighting function:

$$\int_\Gamma R(\mathbf{x})\delta(\mathbf{x}-\mathbf{x}_i)d\Gamma = R(\mathbf{x}_i) = 0 \quad \text{for } i = 1, 2, \ldots, N \tag{2.3}$$

  This requires only a single surface integration per element interaction, resulting in faster matrix assembly, though it yields non-symmetric influence matrices.
- **Galerkin Method:** The residual is minimized over the entire element domain using the chosen shape functions $N_i(\mathbf{x})$ as the weighting functions, ensuring the error is orthogonal to the trial space:

$$\int_\Gamma R(\mathbf{x})N_i(\mathbf{x})d\Gamma = 0 \quad \text{for } i = 1, 2, \ldots, N \tag{2.4}$$

  This requires computationally demanding double integration—over both the source and field elements—but yields symmetric, stable matrices, providing a rigorous test of an LLM's ability to handle complex nested numerical quadratures.

**3. Geometric Discretization: Element Types**

To solve the integral equations, the boundary $\Gamma$ and the continuous variables $u$ and $q$ are discretized using local shape functions $N_j(\xi)$ over a parametric domain $\xi \in [-1, 1]$:

$$u(\xi) \approx \sum_{j=1}^{m} N_j(\xi)u_j, \quad q(\xi) \approx \sum_{j=1}^{m} N_j(\xi)q_j \tag{2.5}$$

The choice of $m$ dictates the element type:

- **Constant Elements ($m = 1$):** The simplest discretization, where the potential and flux are assumed constant across the entire element, and the collocation node is placed at the geometric centroid.

- **Linear Elements ($m = 2$):** The physical variables vary linearly across the element, with nodes placed at the boundary intersections. This introduces a new testing vector for the LLM: the correct handling of flux discontinuities at geometric corners.

*Note on Dimensionality:* While formulated here for 2D domains, this theoretical framework extends directly to the 3D test cases. The fundamental Green's functions and element types (e.g., 3-node surface triangles) are updated accordingly for 3D boundaries, but the underlying BEM methodology remains identical.

By permuting these three axes—such as Direct-Collocation-Constant vs. Indirect-Galerkin-Linear—a comprehensive matrix of BEM methodologies is generated. This taxonomy forms the foundation of the prompting framework, allowing the study to systematically isolate and evaluate an LLM's capacity to handle specific mathematical and algorithmic constraints.

## 2.2 THEORETICALLY DRIVEN METHOD SELECTION

While the taxonomy defined in Section 2.1 presents a combinatorial matrix of potential BEM solvers, systematically prompting a Large Language Model (LLM) to generate every possible formulation across all geometries is computationally exhaustive and often yields mathematically less favorable configurations. Certain mathematical formulations are naturally better suited for specific physical boundary conditions.Therefore, rather than evaluating the models across all permutations as the primary objective, this study predominantly enforced theoretically optimal pairings. The foundation models were guided to utilize specific formulations based on the problem statement, thereby benchmarking their ability to synthesize standard, best-practice engineering code. However, to establish a comparative baseline, the reverse formulations (using the Direct method for Dirichlet problems and the Indirect method for mixed boundary conditions) were also tested on two basic geometries. The findings of these specific tests are also

presented in subsequent sections. The two primary theoretical constraints enforced during the core testing protocol were:

**1. Direct Formulations for Mixed Boundary Conditions**

For domains governed by mixed boundary conditions—where the potential $u$ is known on $\Gamma_1$ and the flux $q$ is known on $\Gamma_2$—the LLMs were prompted to use the Direct formulation. The Direct method explicitly preserves the physical meaning of the boundary variables. When discretized into the system $[H]\{u\} = [G]\{q\}$, mixed conditions are resolved by rearranging columns between the $[H]$ and $[G]$ matrices to isolate all unknown variables into a single vector $\{x\}$.While Indirect formulations can be adapted for mixed conditions, they require solving for non-physical source densities, which introduces additional mathematical complexity when mapping these densities back to physical boundary values. By guiding the LLMs toward the Direct formulation for these cases, the models were tested on their ability to correctly execute standard matrix manipulations ($[A]\{x\} = \{b\}$) to directly yield the physical unknowns.

**2. Indirect Double-Layer Potentials for Dirichlet Problems**

For pure Dirichlet problems—where the potential $u$ is prescribed over the entire boundary $\Gamma$—the LLMs were prompted to utilize the Indirect Double-Layer Potential (DLP) formulation.Mathematically, when the observation point is taken to the boundary, the continuous DLP formulation yields a Fredholm integral equation of the second kind [5]. This is an advantageous property in computational mechanics because the presence of the identity operator generally leads to well-conditioned influence matrices. Solvers built on this formulation tend to exhibit improved numerical stability and favorable convergence behavior, particularly when utilizing iterative algorithms like GMRES [6]. Prompting the LLMs to adopt this formulation tested their capacity to correctly implement the double-layer integration routines while resulting in a more robust algebraic system.

## 2.3 THE PROMPTING PIPELINE AND ITERATIVE FEEDBACK

The code synthesis pipeline was anchored by a standardized zero-shot prompt. While this initial prompt yielded successful and executable BEM solvers on the first attempt, specific combinations of complex geometries, mixed boundary conditions, and theoretical formulations produced mathematical violations—such as sign reversals in fundamental boundary integral expressions.

To address these failure cases and ensure a consistent methodology across all tested domains, an empirical **Iterative Feedback Loop** was established. This pipeline was designed to handle the generation, evaluation, and correction of the BEM solvers through a formalized, four-step protocol:

### Step 1: The Zero-Shot Baseline Prompt

To ensure reproducibility across different foundation models, a modular prompt template was utilized for all initial interactions. The template defined the target geometry, physical boundary conditions, and implementation constraints to maximize the probability of a successful zero-shot generation:

> **Objective:** Generate a complete, runnable Python implementation (single script) of a 2D Boundary Element Method (BEM) solver for [GOVERNING EQUATION].
>
> **Domain & Exact Solution:**
>
> - **Domain:** [SPECIFY GEOMETRY AND BOUNDARY DEFINITIONS]
> - **Exact solution:** [SPECIFY EXACT SOLUTION]
>
> **Boundary Conditions:**
>
> - [SPECIFY BCs, e.g., Dirichlet on Boundary 1, Neumann on Boundary 2]
>
> **Methodology:**
>
> - **Formulation:** [DIRECT / INDIRECT]
> - **Weighting:** [COLLOCATION / GALERKIN]

- **Elements:** [CONSTANT / LINEAR / QUADRATIC]

**Implementation Requirements:**

1. **Discretization:** [SPECIFY RULES, e.g., uniform angular spacing]
2. **Geometry Preprocessing:** Precompute midpoints, normals, lengths. [SPECIFY NORMAL DIRECTIONS]
3. **Quadrature:** Use Gaussian quadrature (order $\geq$ 8). [SPECIFY SINGULARITY HANDLING]
4. **System Assembly:** [SPECIFY SOLVER APPROACH, e.g., Matrix-free GMRES via SciPy LinearOperator or Dense Assembly]
5. **RHS Assembly:** Evaluate known boundary data using the exact solution.
6. **Interior Evaluation:** Use the direct representation formula for interior points.

**Testing & Output Constraints:**

- Evaluate on fixed interior grid: [SPECIFY GRID PARAMETERS]
- Run refinement study for N = [SPECIFY N VALUES].
- Report unknowns, iterations, relative L2 error, and timing metrics.
- Constraints: Use NumPy, SciPy, Numba only. Output only clean runnable code. No explanatory text.

### Step 2: Execution and Error Diagnostic

The initially generated code was compiled and executed. If the code successfully achieved an acceptable relative L2 error against the analytical solution, it bypassed the feedback loop. Basic syntactical or compilation errors were disregarded; the formal evaluation and feedback loop were initiated for codes that successfully executed but failed to produce mathematically valid results. These execution failures manifested as either **Theoretical Violations**—where the LLM dropped mathematical terms —or **Directional/Physical Errors**—where the code produced an unacceptable relative L2 error due to flaws such as sign reversals in normal vectors or incorrect integration weights.

**Step 3: The Iterative Feedback Loop and Termination Criteria**

For codes requiring correction, the execution logs and L2 error metrics were shared back with the LLM. This iterative debugging loop was governed by empirical limits:

1. **Feedback Limit:** The model was allowed a maximum of 5 iterative feedback loops to correct the solver.
2. **Context Reset:** If the model failed to produce a mathematically sound solver after 5 attempts, the conversational context was assumed to be saturated with errors. The chat session was terminated, and the identical prompt was tested in a fresh context window.
3. **Failure Conclusion:** If the fresh session also failed to converge upon an accurate solver within 5 loops, the model was empirically concluded to be incapable of handling that specific BEM configuration.

**Step 4: Algorithmic Acceleration**

Once a functionally accurate solver was obtained—whether on the first zero-shot attempt or after iterative correction—a final prompt was issued. The model was instructed to computationally accelerate the validated code—for example, through Numba vectorization or optimized memory allocation—without breaking the established physics. This allowed the study to capture both the baseline accuracy and the peak computational efficiency of the LLM-generated codes.

## 2.4 EVALUATION METRICS

Evaluating the efficacy of Large Language Models in generating BEM solvers requires assessing both the mathematical accuracy of the resulting code and the efficiency of the generation process. Once a solver is successfully executed, its outputs were benchmarked against known analytical solutions across a fixed internal domain grid.

The performance of the LLMs and their generated solvers was quantified using the following metrics:

- **Relative $L_2$ Error:** The primary metric for verifying mathematical accuracy. The error was evaluated on a fixed grid of internal points to ensure that the boundary integral representations were functioning correctly. The relative $L_2$ error is defined as:

$$E_{L_2} = \sqrt{\frac{\sum_{i=1}^{M} (u_{num,i} - u_{exact,i})^2}{\sum_{i=1}^{M} (u_{exact,i})^2}} \tag{2.6}$$

  where $M$ is the total number of evaluation points, $u_{num,i}$ is the numerical potential calculated by the solver, and $u_{exact,i}$ is the known analytical potential.
- **Mesh Convergence ($h$-refinement):** To verify that the generated solvers exhibited proper asymptotic behavior, a refinement study was executed as part of the prompt constraints. The boundary mesh was uniformly refined—such as from $N = 40$ to $N = 80$ and $N = 160$ elements—to confirm that the $L_2$ error decreased monotonically.
- **Computational Timing:** The execution time was isolated into three distinct phases: Setup Time—encompassing geometry pre-processing and matrix or operator assembly; Solve Time—representing linear system resolution; and Evaluation Time—for post-processing the internal domain grid.
- **Generative Efficiency (Iteration Count):** To evaluate the viability of using foundation models as engineering co-pilots, the number of feedback iterations required to achieve a mathematically valid solver was recorded for each experiment. This provided an empirical measure of the model's zero-shot capabilities versus its reliance on guided correction. We denote this by $I_c$ in this study.

## CODE AVAILABILITY

All Python-based BEM solvers generated by the LLMs for the test cases in this study are available in the supplementary repository:

https://github.com/Pranv45/LLM-Guided-Generation-of-BEM-Solvers

# CHAPTER 3

# LLM CODE SYNTHESIS FOR 2D DOMAINS

## 3.1 INTRODUCTION TO 2D TESTING

This chapter presents the empirical evaluation of LLM-generated BEM solvers applied to two-dimensional (2D) geometries. The code synthesis pipeline is evaluated comparatively across three foundation models: **Gemini Pro**, **ChatGPT (Thinking)**, and **Claude Sonnet**.

The evaluation is structured into two primary phases. Part I assesses the foundation models on basic geometries to verify algorithmic logic and matrix assembly. Part II advances the testing to more complex 2D domains to evaluate the numerical behavior of the generated solvers under compounded geometric and theoretical constraints.

## 3.2 PART I: BASIC 2D GEOMETRIES

### 3.2.1 The Unit Circle

**Problem Definition**

Laplace equation in the circular domain ($r = 1$):

$$\nabla^2 u = 0 \tag{3.1}$$

**Boundary Condition:**

$$u(1, \theta) = \cos(n\theta) \tag{3.2}$$

**Analytical Solution:**

$$u(r, \theta) = r^n \cos(n\theta) \tag{3.3}$$

For this study, $n = 3$ was utilized.

**Scheme 1: Indirect DLP — Collocation — Constant Elements**

**Generative Efficiency:** All three foundation models generated the baseline solver zero-shot ($I_c = 0$), adhering to the requested formulation constraints. ChatGPT and Claude initially implemented block-chunking logic to manage memory during purely NumPy-based matrix-vector operations.

**Computational Performance & Algorithmic Acceleration:** The solvers across all models demonstrated strict monotonic convergence. The initial NumPy-based matrix-free implementations successfully scaled to the finest mesh ($M = 64,000$ elements), achieving a relative $L_2$ error of approximately $1.56 \times 10^{-5}$. However, these pure Python implementations were computationally expensive, requiring total execution times ranging from 171 to 234 seconds depending on the model.

In accordance with the established testing pipeline, the models were subsequently prompted to apply Numba vectorization to accelerate the validated scripts. As shown in Tables 3.1, 3.2, and 3.3, this compilation strategy drastically reduced execution time. For the finest mesh ($M = 64,000$), total execution times collapsed to under 16 seconds across all three models while maintaining the identical $1.56 \times 10^{-5}$ accuracy and single-iteration GMRES convergence characteristic of the Indirect DLP formulation.

Table 3.1: Mesh refinement study of the Gemini Pro Indirect DLP solver using constant collocation on the unit circle.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 16,000 | 6.250e-05 | 0.002 | 1.087 | 0.399 | 1.492 |
| 32,000 | 3.125e-05 | 0.003 | 4.059 | 0.575 | 4.642 |
| 64,000 | 1.563e-05 | 0.015 | 13.695 | 1.014 | 14.727 |

Table 3.2: Mesh refinement study of the ChatGPT Indirect DLP solver using constant collocation on the unit circle.

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 16,000 | 6.250e-05 | 0.002 | 1.020 | 0.280 | 1.302 |
| 32,000 | 3.130e-05 | 0.008 | 3.202 | 0.521 | 3.730 |
| 64,000 | 1.560e-05 | 0.007 | 11.056 | 0.822 | 11.885 |

Table 3.3: Mesh refinement study of the Claude Sonnet Indirect DLP solver using constant collocation on the unit circle.

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 16,000 | 6.250e-05 | 0.001 | 0.900 | 0.286 | 1.190 |
| 32,000 | 3.125e-05 | 0.003 | 3.372 | 0.523 | 3.901 |
| 64,000 | 1.563e-05 | 0.006 | 13.988 | 1.215 | 15.214 |

**Scheme 2: Indirect DLP — Collocation — Linear Elements**

**Generative Efficiency:** ChatGPT and Claude generated the linear element solvers zero-shot ($I_c = 0$). Gemini Pro required one correction cycle ($I_c = 1$) to resolve a broadcasting error in the `bem_matvec` function where collocation node shapes conflicted with quadrature points. Claude's initial implementation was the most sophisticated, incorporating Gaussian quadrature point caching. Following the initial implementations, all three solvers were refactored using Numba JIT compilation (‘@njit(parallel=True)‘) to eliminate $O(N^2)$ memory allocations and leverage multi-threading.

**Computational Performance (Numba Accelerated):** The introduction of Numba dramatically transformed the performance landscape. Prior to Numba acceleration, execution at the finest mesh ($M = 16,000$) was severely bottlenecked by memory allocation during NumPy matrix-vector products, yielding total execution times of 655.03s for Gemini (Relative $L_2$ Error: $1.25 \times 10^{-4}$), 1547.29s for ChatGPT (Relative $L_2$ Error: $2.33 \times 10^{-4}$), and 175.98s for Claude (Relative $L_2$ Error: $5.75 \times 10^{-4}$). Following JIT compilation, these total times collapsed to mere seconds across all models (Tables 3.4,

3.5, and 3.6). While all three models maintained monotonic $O(h)$ convergence, Gemini achieved both the highest computational throughput and the lowest relative $L_2$ error ($1.249 \times 10^{-4}$ at $M = 16,000$). Notably, Gemini's JIT-compiled matrix-vector product executed significantly faster than the competitors (1.8s vs ∼10–13s at $M = 16,000$), demonstrating a highly optimized nested loop structure well-suited for Numba's LLVM backend.

Table 3.4: Mesh refinement study of the Gemini Pro Indirect DLP solver using linear collocation on the unit circle.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 4,000 | 4.984e-04 | 0.002 | 0.138 | 0.147 | 0.287 |
| 8,000 | 2.496e-04 | 0.003 | 0.486 | 0.290 | 0.779 |
| 16,000 | 1.249e-04 | 0.006 | 1.801 | 0.822 | 2.629 |

Table 3.5: Mesh refinement study of the ChatGPT Indirect DLP solver using linear collocation on the unit circle.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 4,000 | 9.300e-04 | 0.006 | 0.897 | 0.322 | 1.225 |
| 8,000 | 4.650e-04 | 0.008 | 2.811 | 0.633 | 3.451 |
| 16,000 | 2.330e-04 | 0.012 | 13.112 | 1.948 | 15.073 |

Table 3.6: Mesh refinement study of the Claude Sonnet Indirect DLP solver using linear collocation on the unit circle.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 4,000 | 2.302e-03 | 0.001 | 0.550 | 0.890 | 1.445 |
| 8,000 | 1.150e-03 | 0.002 | 2.712 | 2.335 | 5.052 |
| 16,000 | 5.749e-04 | 0.004 | 10.759 | 3.818 | 14.584 |

**Scheme 3: Indirect DLP — Galerkin — Linear Elements**

**Generative Efficiency & Feedback Loop:** For this scheme, the initial prompts explicitly constrained the models to utilize Numba vectorization for algorithmic acceleration on the

first attempt. While all three models successfully synthesized executable, compiled code without runtime errors, none produced a mathematically sound solver. Across all three models, the iterative feedback loop was exhausted ($I_c = 5$). Despite corrective prompting regarding irregular convergence, the LLMs failed to identify and correct the underlying theoretical flaws—likely improper quadrature handling for singular and near-singular double integrals inherent to the Galerkin formulation.

**Computational Performance & Numerical Stability:** Due to the explicit Numba constraints, the matrix assembly (Setup) and Evaluation phases were highly accelerated. However, as shown in Tables 3.7, 3.8, and 3.9, all three generated solvers failed to achieve strict monotonic convergence.

The relative $L_2$ error exhibited oscillatory or divergent behavior as the mesh was refined. For example, Gemini's error increased from $2.98 \times 10^{-6}$ at $M = 800$ to $6.23 \times 10^{-6}$ at $M = 1600$, while ChatGPT and Claude exhibited similar non-physical error scaling. This irregular numerical behavior confirms that while foundation models can syntactically structure Numba-accelerated Galerkin double-loops, they lack the intrinsic numerical analysis capabilities required to handle the strongly singular kernel integrations accurately without human intervention.

Table 3.7: Mesh refinement study of the Gemini Pro Indirect DLP solver using linear galerkin on the unit circle.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|:---:|:---:|:---:|:---:|:---:|
| 400 | 6.790e-05 | 0.010 | 0.001 | 0.026 | 0.037 |
| 800 | 2.987e-06 | 0.041 | 0.001 | 0.056 | 0.098 |
| 1,600 | 6.232e-06 | 0.156 | 0.003 | 0.104 | 0.263 |

Table 3.8: Mesh refinement study of the ChatGPT Indirect DLP solver using linear galerkin on the unit circle.

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 400 | 2.280e-04 | 0.077 | 0.001 | 0.065 | 0.144 |
| 800 | 1.480e-04 | 0.271 | 0.002 | 0.091 | 0.364 |
| 1,600 | 8.220e-05 | 0.868 | 0.004 | 0.218 | 1.090 |

Table 3.9: Mesh refinement study of the Claude Sonnet Indirect DLP solver using linear galerkin on the unit circle.

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 400 | 5.314e-05 | 0.038 | 0.001 | 0.024 | 0.064 |
| 800 | 5.998e-05 | 0.163 | 0.001 | 0.066 | 0.232 |
| 1,600 | 3.834e-05 | 0.563 | 0.003 | 0.095 | 0.662 |

**Scheme 4: Direct — Collocation — Linear Elements**

While the Indirect formulation is theoretically the most natural choice for a pure Dirichlet problem, the Direct method was explicitly tested to evaluate how well the foundation models could navigate its distinct mathematical complexities.

However, across all three tested LLMs, the generated solvers consistently failed to converge, yielding relative $L_2$ errors of approximately 1.0 across all evaluated mesh sizes.The models were unable to correctly implement the specialized mathematical handling required for the singular integrals inherent to the Direct formulation. Consequently, none of the foundation models produced a valid numerical solution, even after exhausting the maximum allotted feedback cycles ($I_c = 5$).

**Scheme 5: Spectral Method (Fourier Diagonalization)**

**Generative Efficiency:** When explicitly prompted to exploit the analytical geometric advantages of the unit circle, all three foundation models successfully generated a Spectral BEM solver zero-shot ($I_c = 0$). By utilizing Fourier diagonalization, the models correctly

bypassed standard matrix assembly and GMRES iterative solving, instead computing the solution directly in frequency space using Fast Fourier Transforms (FFTs).

**Computational Performance:** To illustrate the spectral convergence characteristic of this method, the performance metrics in Tables 3.10, 3.11, and 3.12 are restricted to the three coarsest meshes ($M = 100$, 200, and 400). The relative $L_2$ errors across the models decayed exponentially, reaching machine precision ($\approx 10^{-15}$) by $M = 400$. Because the dense linear algebra operations were eliminated, total execution times for all models were reduced to fractions of a second.

Table 3.10: Mesh refinement study of the Gemini Pro Spectral BEM solver on the unit circle.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|
| 100 | 2.148e-15 | 1.000e-04 | 1.800e-03 | 1.900e-03 |
| 200 | 1.857e-15 | 1.000e-04 | 3.100e-03 | 3.200e-03 |
| 400 | 1.462e-15 | 1.000e-04 | 3.300e-03 | 3.400e-03 |

Table 3.11: Mesh refinement study of the ChatGPT Spectral BEM solver on the unit circle.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|
| 100 | 1.039e-05 | 1.000e-03 | 2.220e-02 | 2.320e-02 |
| 200 | 1.982e-10 | 2.000e-03 | 5.110e-02 | 5.130e-02 |
| 400 | 1.015e-15 | 2.000e-03 | 7.450e-02 | 7.470e-02 |

Table 3.12: Mesh refinement study of the Claude Sonnet Spectral BEM solver on the unit circle.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|
| 100 | 1.039e-05 | 1.000e-04 | 4.570e-02 | 6.030e-02 |
| 200 | 1.982e-10 | 1.000e-04 | 8.050e-02 | 8.270e-03 |
| 400 | 7.101e-16 | 1.000e-04 | 1.200e-01 | 1.220e-01 |

### Visual Summary

Figure 3.1 provides a visual summary of the performance for the finest meshes evaluated across all formulations in the circular domain, illustrating the time-to-accuracy trade-offs and capability limits discussed in the preceding sections. The plot is on a log-log scale.

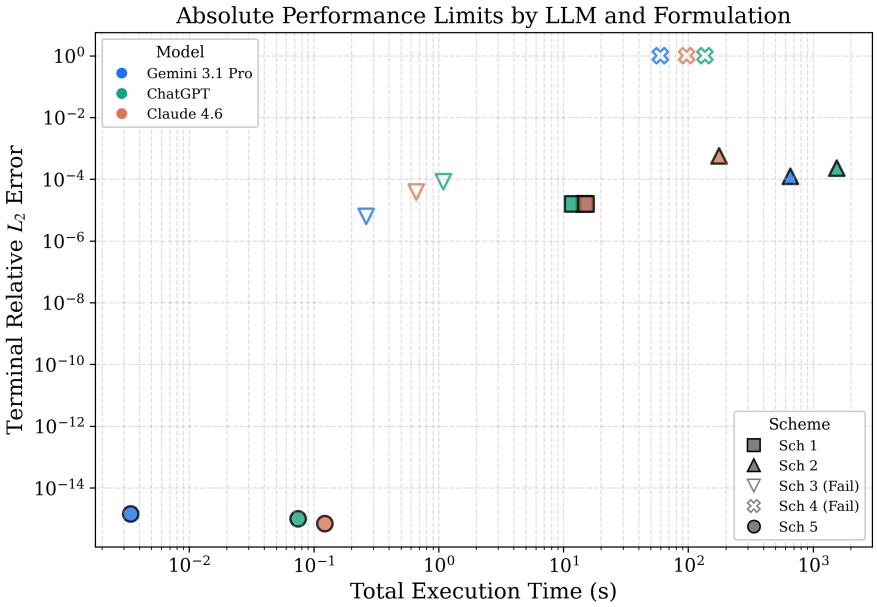



Figure 3.1: $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for Circle

### 3.2.2 Annulus

**Problem Definition**

Laplace equation in an annular domain with mixed boundary conditions.

$$\nabla^2 u = 0 \quad \text{in } \Omega = \{(r, \theta) \mid 1 < r < 2\} \tag{3.4}$$

**Boundary Conditions:**

**Dirichlet (Inner Boundary):**

$u(1, \theta) = 2\cos(3\theta)$

**Neumann (Outer Boundary):**

$\frac{\partial u}{\partial n}(2, \theta) = \frac{189}{16}\cos(3\theta)$

**Analytical Solution:**

$$u(r, \theta) = \left(r^3 + r^{-3}\right) \cos(3\theta) \tag{3.5}$$

**Scheme 1: Direct Formulation — Collocation — Constant Elements**

**Generative Efficiency:** ChatGPT generated the Direct Constant Collocation solver zero-shot ($I_c = 0$), autonomously integrating a high-order 32-point Gauss-Legendre quadrature scheme to handle the multiply-connected curved domain. Gemini Pro and Claude Sonnet both required one correction cycle ($I_c = 1$). Gemini initially failed to converge to the correct mathematical solution, yielding a flatlined relative error. Claude's initial zero-shot failure was driven by incorrect handling of singular self-terms and imprecise normal vector approximations; however, following feedback, it successfully implemented analytical regularization and evaluated exact outward normals at each quadrature point.

**Computational Performance:** The corrected formulations demonstrated stable algebraic convergence across all models (Tables 3.13, 3.14, and 3.15). Gemini and Claude significantly outperformed ChatGPT in accuracy, achieving terminal relative $L_2$ errors of $5.778 \times 10^{-6}$ and $3.152 \times 10^{-6}$ respectively at $M = 5120$, with overall convergence rates exceeding 2.4. In contrast, ChatGPT yielded an error of $3.168 \times 10^{-4}$ with a convergence rate of 1.08. The matrix-free GMRES implementations were highly well-conditioned across all models. Gemini demonstrated the highest overall computational throughput, executing the finest mesh ($M = 5120$) in 5.50 seconds compared to Claude's 7.41 seconds and ChatGPT's 68.62 seconds.

Table 3.13: Mesh refinement study of the Gemini Pro Direct Constant Collocation BEM solver on the annular domain.

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,280 | 1.673e-04 | 0.032 | 0.151 | 0.085 | 0.268 |
| 2,560 | 2.342e-05 | 0.239 | 0.731 | 0.228 | 1.198 |
| 5,120 | 5.778e-06 | 0.879 | 4.194 | 0.431 | 5.504 |

Table 3.14: Mesh refinement study of the ChatGPT Direct Constant Collocation BEM solver on the annular domain.

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,280 | 1.329e-03 | 0.851 | 2.668 | 1.573 | 5.092 |
| 2,560 | 6.433e-04 | 3.589 | 10.695 | 3.079 | 17.363 |
| 5,120 | 3.168e-04 | 15.634 | 46.311 | 6.673 | 68.618 |

Table 3.15: Mesh refinement study of the Claude Sonnet Direct Constant Collocation BEM solver on the annular domain.

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,280 | 1.461e-04 | 0.082 | 0.002 | 0.224 | 0.308 |
| 2,560 | 1.316e-05 | 1.095 | 0.005 | 0.604 | 1.704 |
| 5,120 | 3.152e-06 | 5.709 | 0.024 | 1.681 | 7.414 |

**Scheme 2: Direct Formulation — Collocation — Linear Elements**

**Generative Efficiency:** All three models required one correction cycle ($I_c = 1$) to achieve the theoretical $O(h^2)$ convergence expected for linear elements, as their zero-shot attempts ($I_c = 0$) uniformly yielded sub-optimal $O(h)$ convergence rates. Gemini Pro's initial accuracy degradation stemmed from an improper assumption regarding the free term $c(\xi)$ at elemental junctions, failing to calculate it based on the interior angle of the discretized polygon. Claude Sonnet suffered from a logic bug in the matrix assembly process, erroneously skipping numerical quadrature accumulation for weakly-singular self-terms in the Single-Layer Potential matrix. ChatGPT exhibited

similar singular integration issues; however, upon correction, it autonomously pivoted away from a matrix-free GMRES architecture. Instead, ChatGPT implemented explicit dense matrix assembly, a direct linear solver (`scipy.linalg.solve`), and a dynamic adaptive quadrature scheme (32-point far-field vs. 128-point near-field).

**Computational Performance:** Following the single feedback cycle, all three models successfully restored optimal algebraic convergence, achieving terminal relative $L_2$ errors in the $4 \times 10^{-6}$ to $6 \times 10^{-6}$ range at $M = 5120$ (Tables 3.16, 3.17 and 3.18). Gemini and Claude maintained their matrix-free GMRES implementations, converging in 2 and 3 iterations, respectively. Gemini demonstrated the highest computational throughput, completing the finest mesh in 4.20 seconds, followed closely by Claude at 6.88 seconds. Conversely, ChatGPT's architectural pivot to explicit dense matrix assembly and intensive near-field quadrature incurred a significant computational penalty, elevating its total execution time at $M = 5120$ to 26.04 seconds.

Table 3.16: Mesh refinement study of the Gemini Pro Direct Linear Collocation BEM solver on the annular domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,280 | 7.767e-05 | 0.057 | 0.158 | 0.073 | 0.288 |
| 2,560 | 1.790e-05 | 0.214 | 0.787 | 0.199 | 1.200 |
| 5,120 | 4.470e-06 | 0.884 | 2.989 | 0.327 | 4.200 |

Table 3.17: Mesh refinement study of the ChatGPT Direct Linear Collocation BEM solver on the annular domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,280 | 9.772e-05 | 0.979 | 0.061 | 1.721 | 2.761 |
| 2,560 | 2.452e-05 | 4.065 | 0.351 | 3.426 | 7.842 |
| 5,120 | 6.169e-06 | 16.824 | 2.203 | 7.011 | 26.038 |

Table 3.18: Mesh refinement study of the Claude Sonnet Direct Linear Collocation BEM solver on the annular domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,280 | 1.003e-04 | 0.162 | 0.002 | 0.481 | 0.645 |
| 2,560 | 2.510e-05 | 0.702 | 0.007 | 1.246 | 1.955 |
| 5,120 | 6.274e-06 | 2.964 | 0.029 | 3.889 | 6.882 |

**Scheme 3: Direct Formulation — Galerkin — Linear Elements**

**Generative Efficiency:** The double-integration architecture required by the Galerkin method posed a significant challenge across all models. ChatGPT (Thinking) suffered a hard capability failure ($I_c = 5$), overcomplicating the system by improperly mixing Galerkin integration techniques (such as Duffy transformations) into the architecture. Gemini Pro and Claude Sonnet successfully implemented the double integrations zero-shot but failed to analytically resolve the logarithmic singularity for overlapping test and source elements, relying instead on standard Gauss-Legendre points which degraded their accuracy. Gemini corrected this with a single feedback cycle ($I_c = 1$) by implementing a logarithmic-weighted quadrature scheme. Claude required two cycles ($I_c = 2$) after an initial destabilizing attempt, ultimately succeeding by deriving exact analytical antiderivatives for coincident elements and deploying an intensive 32-point Gauss-Legendre rule for adjacent near-singular nodes.

**Computational Performance:** Gemini and Claude successfully restored optimal $O(h^2)$ algebraic convergence following their respective feedback cycles (Tables 3.19 and 3.20). Both matrix-free solvers exhibited excellent conditioning, converging in exactly 2 GMRES iterations. Gemini achieved a relative $L_2$ error of $4.575 \times 10^{-6}$ at $M = 2560$ with a total execution time of 43.45 seconds. Claude achieved a slightly lower error of $3.688 \times 10^{-6}$ at the same resolution, but its complex analytical antiderivatives imposed a severe computational penalty, driving its execution time to 461.35 seconds. In stark contrast, ChatGPT's architectural confusion resulted in a catastrophically ill-posed system

that actively diverged upon mesh refinement, yielding a massive relative error of 1700% ($1.70 \times 10^{1}$) at $M = 2560$.

Table 3.19: Mesh refinement study of the Gemini Pro Direct Linear Galerkin BEM solver on the annular domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---|---|---|---|---|
| 640 | 5.896e-05 | 2.285 | 0.005 | 0.139 | 2.429 |
| 1,280 | 1.593e-05 | 7.933 | 0.010 | 0.202 | 8.145 |
| 2,560 | 4.575e-06 | 42.773 | 0.039 | 0.638 | 43.450 |

Table 3.20: Mesh refinement study of the Claude Sonnet Direct Linear Galerkin BEM solver on the annular domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---|---|---|---|---|
| 640 | 5.541e-05 | 6.416 | 20.091 | 0.512 | 27.019 |
| 1,280 | 1.416e-05 | 25.948 | 83.867 | 1.031 | 110.846 |
| 2,560 | 3.688e-06 | 114.051 | 345.152 | 2.149 | 461.352 |

**Exploratory Results: Indirect Formulations for Mixed Boundary Conditions**

To test the limits of the prompting constraints in Section 2.2, the annulus problem with mixed boundary conditions was also attempted using indirect formulations. Although mathematically valid, these formulations are less natural for mixed problems because they introduce fictitious density unknowns instead of directly preserving the physical boundary variables.

The models did not produce reliable convergent solvers in this setting. As shown in Table 3.21, ChatGPT and Claude yielded relative $L_2$ errors above unity, while Gemini Pro achieved the best indirect result, with a relative $L_2$ error of $1.30 \times 10^{-2}$ and an observed convergence rate of 0.85 for the indirect DLP linear-collocation scheme.

Table 3.21: Performance for indirect formulations with mixed boundary conditions.

| Formulation | Model | Total (s) | Rel $L_2$ Error | Conv. Rate |
|---|---|---|---|---|
| Indirect DLP (Linear Collocation) | Gemini Pro | 1.27 | 1.30e-02 | 0.84 |
| | ChatGPT (Thinking) | 54.63 | 1.25e+00 | 0.01 |
| | Claude Sonnet | 40.09 | 1.12e+00 | 0.06 |
| Indirect Combined (Linear Collocation) | Gemini Pro | 14.14 | 1.41e-01 | -0.05 |
| | ChatGPT (Thinking) | 380.82 | 1.25e+00 | 0.00 |
| | Claude Sonnet | 40.09 | 1.05e+00 | 0.00 |

### Visual Summary

Figure 3.2 provides a visual summary of the performance for the finest meshes evaluated across all formulations in the annular domain, illustrating the time-to-accuracy trade-offs and capability limits discussed in the preceding sections. The plot is on a log-log scale.

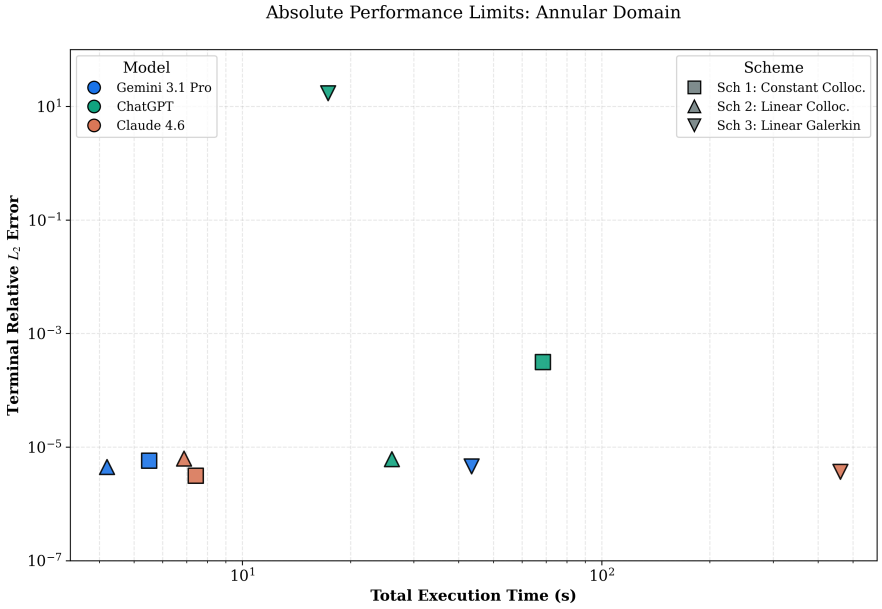



Figure 3.2: $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for Annulus

### 3.2.3 Rectangle

**Problem Definition**

Laplace equation in the rectangular domain with mixed boundary conditions.

$$\nabla^2 u = 0 \quad \text{in } \Omega = [0, 2] \times [0, 1] \tag{3.6}$$

**Dirichlet Boundary Conditions:**

- $u(0, y) = 0$
- $u(2, y) = \cos(\pi y)$

**Neumann Boundary Conditions:**

- $\frac{\partial u}{\partial y}(x, 0) = 0$
- $\frac{\partial u}{\partial y}(x, 1) = 0$

**Analytical Solution:**

$$u(x, y) = \frac{\sinh(\pi x)}{\sinh(2\pi)} \cos(\pi y) \tag{3.7}$$

**Scheme 1: Direct Formulation — Collocation — Constant Elements**

**Generative Efficiency:** The introduction of sharp 90-degree corners exposed a critical limitation across all models. Despite exhausting the maximum allowed feedback cycles ($I_c = 5$), ChatGPT, Claude, and Gemini all failed to generate a converging solver. The core failure was identical across the board: the models stubbornly applied a naive midpoint quadrature to the boundary integrals. They failed to autonomously recognize that normal vectors are undefined and fluxes are weakly singular at domain vertices, completely missing the need for specialized geometric treatments like discontinuous elements or analytical corner limits.

**Computational Performance:** As detailed in Table 3.22, mesh refinement yielded absolutely no improvement; overall convergence rates flatlined at exactly 0.00 across all models up to $M = 2560$. Terminal relative $L_2$ errors stalled at roughly 10% ($1.0 \times 10^{-1}$) for ChatGPT, 100% ($1.0 \times 10^{0}$) for Claude, and 200% ($2.0 \times 10^{0}$) for Gemini.

Table 3.22: Metrics at the finest mesh ($M = 2560$)

| Model | Feedback Cycles ($I_c$) | Terminal Rel $L_2$ Error | Convergence Rate |
|---|---|---|---|
| ChatGPT (Thinking) | 5 (Max) | $\sim 1.0 \times 10^{-1}$ | 0.00 |
| Claude Sonnet | 5 (Max) | $\sim 1.0 \times 10^{0}$ | 0.00 |
| Gemini Pro | 5 (Max) | $\sim 2.0 \times 10^{0}$ | 0.00 |

**Scheme 2: Direct Formulation — Collocation — Linear Elements**

**Generative Efficiency:** Despite successfully constructing the baseline continuous linear element architecture, all three models failed to synthesize a valid solver within the maximum allowed feedback cycles ($I_c = 5$). Echoing the failures observed in the constant element formulation, the models neglected the mathematical implications of the 90-degree corners. Specifically, they erroneously hardcoded a uniform free term of $c(\xi) = 0.5$ across all nodes—ignoring the geometric requirement of $c(\xi) = 0.25$ at orthogonal vertices.

**Computational Performance:** As detailed in Table 3.23, the convergence rates for all models flatlined at 0.00 across all evaluated mesh densities up to $M = 2560$. Terminal relative $L_2$ errors stalled at approximately 45% ($4.5 \times 10^{-1}$) for ChatGPT, while both Claude and Gemini Pro experienced massive error propagation approaching 100% ($1.0 \times 10^{0}$).

Table 3.23: Metrics at the finest mesh ($M = 2560$)

| Model | Feedback Cycles ($I_c$) | Terminal Rel $L_2$ Error | Convergence Rate |
|---|---|---|---|
| ChatGPT | 5 (Max) | $\sim 4.5 \times 10^{-1}$ | 0.00 |
| Claude | 5 (Max) | $\sim 1.0 \times 10^{0}$ | 0.00 |
| Gemini Pro | 5 (Max) | $\sim 1.0 \times 10^{0}$ | 0.00 |

**Scheme 3: Direct Formulation — Galerkin — Linear Elements**

**Generative Efficiency:** All three models exhausted the maximum feedback limit ($I_c = 5$) without producing a converging solver. While they successfully assembled the baseline

Galerkin double-integration framework, they all failed for one primary mathematical reason: the inability to handle discontinuous normal fluxes at the domain's 90-degree corners. By erroneously enforcing strict nodal continuity across these sharp vertices, the models created fundamentally ill-posed system matrices.

**Computational Performance:** Mesh convergence flatlined at exactly zero across all models. Terminal relative $L_2$ errors stagnated at approximately 46% for ChatGPT and near 100% for both Claude and Gemini Pro.

Furthermore, the unoptimized double-integration loops natively generated by ChatGPT proved highly inefficient, requiring over nine minutes (547 seconds) simply to assemble the matrix for a minimal 640-element mesh.

Table 3.24: Metrics at the finest mesh ($M = 640$)

| **Model** | **Feedback Cycles ($I_c$)** | **Terminal Rel $L_2$ Error** | **Convergence Rate** |
|---|---|---|---|
| ChatGPT | 5 (Max) | $\sim 4.6 \times 10^{-1}$ | 0.00 |
| Claude | 5 (Max) | $\sim 1.0 \times 10^{0}$ | 0.00 |
| Gemini Pro | 5 (Max) | $\sim 1.0 \times 10^{0}$ | 0.00 |

### 3.2.4 Triangle

**Problem Definition**

Laplace equation in the right triangular domain with pure Dirichlet conditions

$$\nabla^2 u = 0 \qquad \text{in } \Omega = \{(x, y) \in \mathbb{R}^2 \mid x \geq 0,\ y \geq 0,\ x + y \leq 1\}$$

**Boundary condition (pure Dirichlet)**:

$$u(x, y) = \sin(\pi x) \cosh(\pi y) \qquad \text{on } \Gamma \tag{3.8}$$

**Analytical solution**:

$$u_{\text{exact}}(x, y) = \sin(\pi x) \cosh(\pi y) \tag{3.9}$$

**Scheme 1: Indirect Combined Potential — Collocation — Constant Elements**

**Generative Efficiency:** All three models successfully generated functional solvers zero-shot ($I_c = 0$), correctly implementing the indirect combined potential formulation (DLP + $\alpha$ SLP). A key factor in this universal success was the autonomous integration of cosine node clustering toward the vertices, which effectively resolved singular behavior at the geometric corners. Each model displayed distinct numerical strengths: GPT integrated Numba JIT with SciPy's `LinearOperator` for memory efficiency, Claude implemented automated empirical convergence rate calculations and JIT warm-up phases, and Gemini optimized inner loops by precomputing mapped Gauss quadrature points and element lengths.

**Computational Performance:** The formulations yielded highly well-conditioned systems across all models (Tables 3.25, 3.26, and 3.27). The matrix-free GMRES solvers demonstrated exceptional stability; GPT and Gemini achieved perfect mesh-independent convergence with exactly 13 iterations, while Claude converged within 22 to 29 iterations. Gemini achieved the highest numerical accuracy, reaching a terminal relative $L_2$ error of $7.77 \times 10^{-6}$ at $M = 1200$. In terms of throughput, Gemini and GPT were the most efficient, completing the finest mesh execution in 1.75 and 2.25 seconds, respectively, outperforming Claude's 4.53 seconds.

Table 3.25: Mesh refinement study of the Gemini Pro Indirect Combined Potential Constant Collocation BEM solver on the triangular domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|:---:|:---:|:---:|:---:|:---:|
| 300 | 1.423e-04 | 0.008 | 0.175 | 0.055 | 0.238 |
| 600 | 3.008e-05 | 0.016 | 0.477 | 0.080 | 0.573 |
| 1,200 | 7.774e-06 | 0.018 | 1.580 | 0.152 | 1.751 |

Table 3.26: Mesh refinement study of the ChatGPT Indirect Combined Potential Constant Collocation BEM solver on the triangular domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 300 | 1.509e-04 | 1.000e-04 | 0.132 | 0.049 | 0.181 |
| 600 | 4.784e-05 | 1.000e-04 | 0.508 | 0.098 | 0.605 |
| 1,200 | 2.250e-05 | 0.001 | 2.049 | 0.201 | 2.251 |

Table 3.27: Mesh refinement study of the Claude Sonnet Indirect Combined Potential Constant Collocation BEM solver on the triangular domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 300 | 1.442e-04 | 1.000e-04 | 0.361 | 0.068 | 0.429 |
| 600 | 4.064e-05 | 1.000e-04 | 1.157 | 0.106 | 1.263 |
| 1,200 | 1.828e-05 | 0.001 | 4.304 | 0.220 | 4.525 |

**Scheme 2: Indirect DLP — Collocation — Linear Elements**

**Generative Efficiency:**

All three models generated functional solvers zero-shot ($I_c = 0$), autonomously applying cosine node clustering to resolve corner singularities. Each employed distinct optimizations: Claude flattened mesh arrays for Numba JIT and bypassed self-element integrations – as the DLP principal value is zero on straight elements. Gemini Pro minimized inner-loop overhead by precomputing quadrature coordinates, Jacobians, and shape functions. GPT coupled a custom $C^0$ mapping array with SciPy's `LinearOperator` for efficient matrix-free execution.

**Computational Performance:** The resulting solvers were exceptionally accurate and well-conditioned (Tables 3.28, 3.29, and 3.30). All three models demonstrated mathematically perfect quadratic convergence ($O(h^2)$), achieving near-identical terminal relative $L_2$ errors of approximately $1.29 \times 10^{-5}$ at $M = 1200$ nodes. The matrix-free GMRES solvers exhibited high stability; Gemini Pro and GPT required exactly 13

iterations across all mesh densities, while Claude maintained tight convergence between 23 and 27 iterations. Gemini Pro delivered the highest computational throughput, completing the finest mesh resolution in just 0.48 seconds.

Table 3.28: Mesh refinement study of the Gemini Pro Indirect DLP Linear Collocation BEM solver on the triangular domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 300 | 2.048e-04 | 0.005 | 0.030 | 0.012 | 0.047 |
| 600 | 5.137e-05 | 0.009 | 0.115 | 0.023 | 0.147 |
| 1200 | 1.295e-05 | 0.017 | 0.421 | 0.046 | 0.484 |

Table 3.29: Mesh refinement study of the ChatGPT Indirect DLP Linear Collocation BEM solver on the triangular domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 300 | 2.048e-04 | 0.000 | 0.060 | 0.023 | 0.084 |
| 600 | 5.137e-05 | 0.001 | 0.235 | 0.044 | 0.280 |
| 1200 | 1.295e-05 | 0.001 | 0.905 | 0.087 | 0.993 |

Table 3.30: Mesh refinement study of the Claude Sonnet Indirect DLP Linear Collocation BEM solver on the triangular domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 300 | 2.052e-04 | 0.000 | 0.073 | 0.015 | 0.088 |
| 600 | 5.129e-05 | 0.000 | 0.300 | 0.029 | 0.329 |
| 1200 | 1.282e-05 | 0.001 | 1.209 | 0.061 | 1.271 |

**Scheme 3: Indirect Combined Potential — Collocation — Linear Elements**

**Generative Efficiency:** All three models generated functional solvers zero-shot, condensing the combined-field formulation into a well-posed $M \times M$ system and applying cosine node clustering. The primary difference was their treatment of the weakly singular Single Layer Potential (SLP) on self-elements. GPT implemented analytical closed-form integration. Claude applied a Duffy transformation, which

incorrectly used standard Gauss quadrature for a logarithmic singularity instead of a $1/r$ singularity. Gemini Pro used a distance-based geometric cutoff ('if r > 1e-14'), bypassing explicit singular integration.

**Computational Performance:** The combined formulation yielded well-conditioned systems; GMRES iteration counts remained stable (21–27) despite an eightfold increase in system size (Tables 3.31, 3.32, and 3.33 ). Using analytical integration, GPT was the only model to achieve quadratic convergence ($O(h^2)$), yielding the lowest terminal relative $L_2$ error of $5.978 \times 10^{-6}$ at $M = 2400$. Claude's integration approach resulted in a sub-optimal convergence rate of $O(h^{1.6})$ and a terminal error of $1.557 \times 10^{-5}$. Gemini Pro's geometric cutoff yielded the highest error ($3.429 \times 10^{-5}$). Despite these accuracy differences, the Numba-accelerated, matrix-free executions resulted in comparable computational throughput across all three models (~20–23 seconds at $M = 2400$).

Table 3.31: Mesh refinement study of the Gemini Pro Indirect Combined Potential Linear Collocation BEM solver on the triangular domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---|---|---|---|---|
| 600 | 2.025e-04 | 0.055 | 1.174 | 0.159 | 1.388 |
| 1200 | 7.918e-05 | 0.273 | 4.548 | 0.243 | 5.064 |
| 2400 | 3.429e-05 | 0.652 | 19.346 | 0.681 | 20.680 |

Table 3.32: Mesh refinement study of the ChatGPT Indirect Combined Potential Linear Collocation BEM solver on the triangular domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---|---|---|---|---|
| 600 | 9.557e-05 | 0.015 | 1.353 | 0.145 | 1.513 |
| 1200 | 2.391e-05 | 0.027 | 4.823 | 0.257 | 5.107 |
| 2400 | 5.978e-06 | 0.043 | 19.525 | 0.516 | 20.084 |

Table 3.33: Mesh refinement study of the Claude Sonnet Indirect Combined Potential Linear Collocation BEM solver on the triangular domain.

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---|---|---|---|---|
| 600 | 1.313e-04 | 0.010 | 1.450 | 0.170 | 1.630 |
| 1200 | 4.242e-05 | 0.020 | 7.500 | 0.310 | 7.830 |
| 2400 | 1.557e-05 | 0.030 | 22.480 | 0.600 | 23.110 |

### Visual Summary

Figure 3.3 provides a visual summary of the performance for the finest meshes evaluated across all formulations in the triangular domain, illustrating the time-to-accuracy trade-offs and capability limits discussed in the preceding sections. The plot is on a log-log scale.

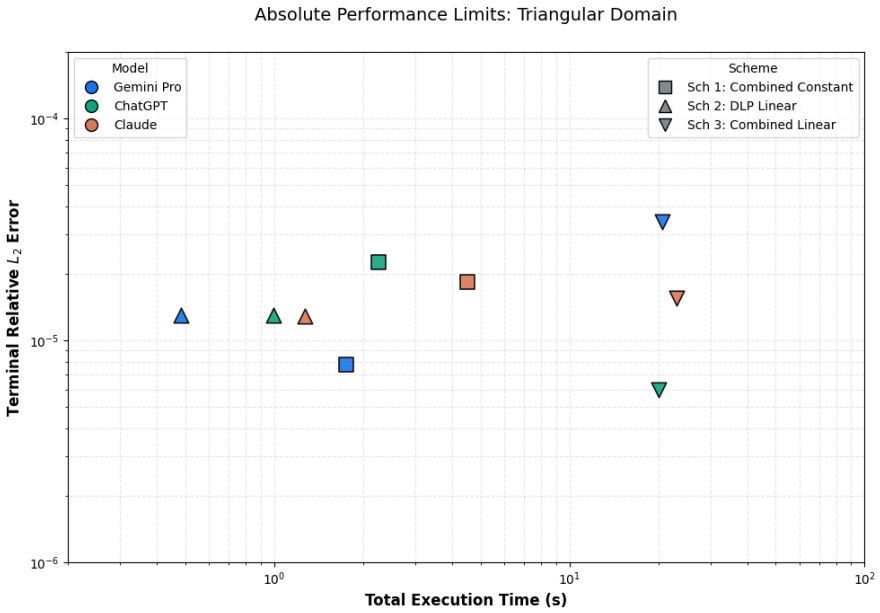



Figure 3.3: $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for Triangle

### 3.2.5 Ellipse

**Problem Definition**

Laplace equation in an elliptical domain with pure Dirichlet conditions

$$\nabla^2 u = 0 \qquad \text{in } \Omega = \left\{ (x, y) \in \mathbb{R}^2 \;\middle|\; \frac{x^2}{a^2} + \frac{y^2}{b^2} < 1 \right\}$$

**Boundary condition (pure Dirichlet)**:

$$u(x, y) = x^2 - y^2 \qquad \text{on } \Gamma = \{(a\cos\theta, b\sin\theta) \mid \theta \in [0, 2\pi)\} \tag{3.10}$$

**Analytical solution**:

$$u_{\text{exact}}(x, y) = x^2 - y^2 \tag{3.11}$$

**Scheme 1: Indirect Double Layer Potential — Collocation — Constant Elements**

**Generative Efficiency:** Gemini Pro and ChatGPT successfully implemented the formulation zero-shot ($I_c = 0$), while Claude required a minor self-correction to fix an initial kernel multiplication error. All three models demonstrated sophisticated high-performance computing practices tailored for massive scaling. They heavily utilized Numba's JIT compilation and multi-threading (`prange`) for the dense matrix-vector products. Crucially, they implemented memory-efficient chunked vectorization strategies for the interior evaluations to prevent memory overflow at large scales. ChatGPT notably enforced memory-contiguous arrays (`np.ascontiguousarray`) to maximize CPU cache efficiency, while Claude integrated a robust geometric orientation check to ensure outward-facing normal vectors.

**Computational Performance:** The matrix-free GMRES solvers exhibited exceptional stability across all models, converging in an extraordinarily low 3 to 4 iterations even as the system size scaled to a massive 64,000 degrees of freedom (Tables 3.34, 3.35, and 3.36). The solvers demonstrated mathematically perfect linear convergence ($O(h)$), achieving near-identical terminal relative $L_2$ errors of approximately $1.54 \times 10^{-5}$ at

$M = 64,000$. While numerical accuracy was consistent across the models, the optimized Numba architectures yielded varying computational throughput. ChatGPT was the most efficient, executing the finest mesh resolution in 14.97 seconds, followed by Claude at 25.83 seconds and Gemini Pro at 34.36 seconds.

Table 3.34: Mesh refinement study of the Gemini Pro Indirect DLP Constant Collocation BEM solver on the elliptical domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 16,000 | 6.135e-05 | 0.075 | 1.015 | 5.753 | 6.843 |
| 32,000 | 3.067e-05 | 0.276 | 4.946 | 11.890 | 17.112 |
| 64,000 | 1.533e-05 | 0.555 | 14.800 | 19.006 | 34.361 |

Table 3.35: Mesh refinement study of the ChatGPT Indirect DLP Constant Collocation BEM solver on the elliptical domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 16,000 | 6.179e-05 | 0.002 | 0.444 | 0.872 | 1.319 |
| 32,000 | 3.089e-05 | 0.002 | 1.987 | 1.900 | 3.889 |
| 64,000 | 1.545e-05 | 0.007 | 10.588 | 4.378 | 14.973 |

Table 3.36: Mesh refinement study of the Claude Indirect DLP Constant Collocation BEM solver on the elliptical domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 16,000 | 6.155e-05 | 0.003 | 1.697 | 0.564 | 2.264 |
| 32,000 | 3.077e-05 | 0.003 | 6.973 | 1.426 | 8.401 |
| 64,000 | 1.538e-05 | 0.010 | 23.893 | 1.924 | 25.827 |

**Scheme 2: Indirect Double Layer Potential — Collocation — Linear Elements**

**Generative Efficiency:** Gemini Pro and Claude generated functional solvers zero-shot ($I_c = 0$). ChatGPT required two feedback cycles ($I_c = 2$) to correct inconsistent linear-element collocation discretization, including corrupted Numba matvec logic and an incorrect treatment/sign of the DLP boundary jump term. Gemini Pro and ChatGPT

used exact polygonal angles for the free-term and bypassed quadrature for adjacent self-elements. Claude hardcoded the free-term to 0.5 and unnecessarily applied 8-point Gauss quadrature to the self-elements, increasing computational overhead.

**Computational Performance:** The matrix-free GMRES solvers demonstrated perfect stability, converging in exactly 4 iterations across all models (Tables 3.37, 3.38, and 3.39). All solvers exhibited a linear convergence rate ($O(h)$). Terminal relative $L_2$ errors at $M = 64,000$ were approximately $6.1 \times 10^{-5}$ for Gemini Pro and ChatGPT, and $3.08 \times 10^{-5}$ for Claude. Gemini Pro was the most efficient (116.06 s), while the unoptimized self-element integration in Claude and ChatGPT led to significantly higher execution times (~400 s).

Table 3.37: Mesh refinement study of the Gemini Pro Indirect DLP Linear Collocation BEM solver on the Elliptical domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 16,000 | 2.454e-04 | 0.219 | 5.115 | 0.437 | 5.771 |
| 32,000 | 1.227e-04 | 0.264 | 25.976 | 0.996 | 27.236 |
| 64,000 | 6.134e-05 | 1.086 | 112.459 | 2.512 | 116.057 |

Table 3.38: Mesh refinement study of the ChatGPT Indirect DLP Linear Collocation BEM solver on the Elliptical domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 16,000 | 2.473e-04 | 0.311 | 22.717 | 0.715 | 23.742 |
| 32,000 | 1.236e-04 | 0.425 | 100.839 | 1.391 | 102.656 |
| 64,000 | 6.181e-05 | 0.790 | 395.865 | 2.362 | 399.018 |

Table 3.39: Mesh refinement study of the Claude Indirect DLP Linear Collocation BEM solver on the Elliptical domain.

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 16,000 | 1.231e-04 | 0.002 | 14.241 | 6.873 | 21.116 |
| 32,000 | 6.154e-05 | 0.003 | 59.586 | 11.235 | 70.824 |
| 64,000 | 3.077e-05 | 0.007 | 383.494 | 23.886 | 407.387 |

**Scheme 3: Indirect Double Layer Potential — Galerkin — Linear Elements**

**Generative Efficiency:** All models generated valid solvers zero-shot ($I_c = 0$). Due to the computational cost of double-integration, their initial implementations required explicit acceleration prompts. To optimize execution: Gemini Pro factored trial-density interpolation out of nested loops; ChatGPT flattened quadrature arrays and precomputed source densities; Claude precomputed global quadrature coordinates.

**Computational Performance:** While the solvers were stable (converging in 12–14 iterations), all models failed to correctly handle near-singular integration for adjacent elements sharing a vertex. Gemini Pro and ChatGPT applied standard 4-point Gauss quadrature, and Claude used a distance cutoff (`r2 < 1.0e-28`). These integration errors limited spatial accuracy to a strict linear convergence rate ($O(h)$), missing the higher-order convergence expected from a Galerkin scheme.

The acceleration prompts significantly improved throughput. For the 32,000-element mesh, Gemini Pro's total execution time dropped from 254.52 s to 108.33 s, Claude's from 748.89 s to 501.08 s, and ChatGPT's from 646.56 s to 576.17 s.

Table 3.40: Mesh refinement study of the Gemini Pro Indirect DLP Linear Galerkin BEM solver on the Elliptical domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---:|---:|---:|---:|---:|
| 8,000 | 9.913e-06 | 0.089 | 4.866 | 0.111 | 5.066 |
| 16,000 | 4.956e-06 | 0.142 | 26.872 | 0.196 | 27.209 |
| 32,000 | 2.478e-06 | 0.353 | 107.587 | 0.391 | 108.331 |

Table 3.41: Mesh refinement study of the ChatGPT Indirect DLP Linear Galerkin BEM solver on the Elliptical domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---:|---:|---:|---:|---:|
| 8,000 | 9.989e-06 | 0.242 | 24.467 | 0.212 | 24.921 |
| 16,000 | 4.994e-06 | 0.604 | 122.356 | 0.828 | 123.789 |
| 32,000 | 2.497e-06 | 1.602 | 573.153 | 1.415 | 576.170 |

Table 3.42: Mesh refinement study of the Claude Indirect DLP Linear Galerkin BEM solver on the Elliptical domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---:|---:|---:|---:|---:|
| 8,000 | 1.853e-06 | 0.011 | 29.876 | 2.468 | 32.355 |
| 16,000 | 9.264e-07 | 0.015 | 114.028 | 4.608 | 118.651 |
| 32,000 | 4.632e-07 | 0.028 | 487.510 | 13.543 | 501.081 |

### Visual Summary

Figure 3.4 provides a visual summary of the performance for the finest meshes evaluated across all formulations in the elliptical domain, illustrating the time-to-accuracy trade-offs and capability limits discussed in the preceding sections. The plot is on a log-log scale.

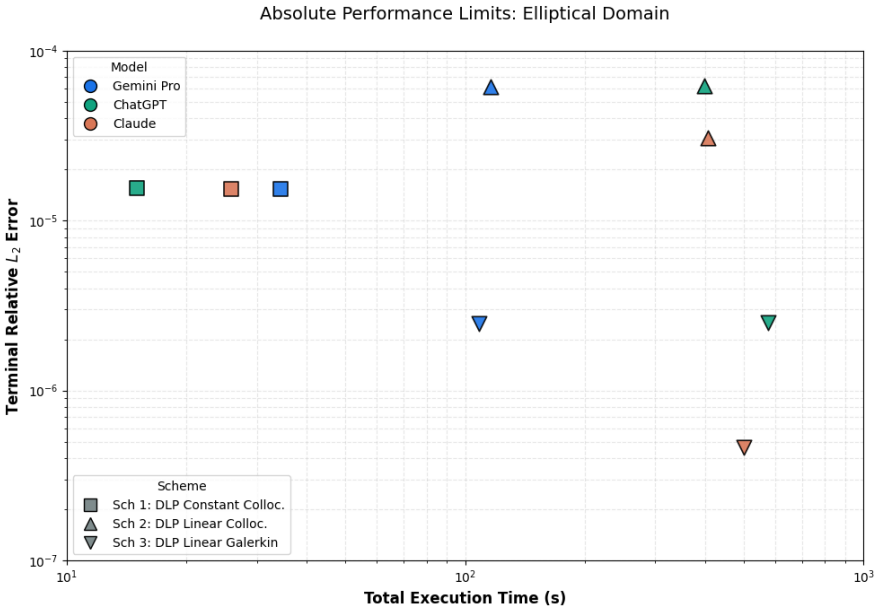



Figure 3.4: $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for Ellipse

## 3.3 PART II: COMPLEX 2D GEOMETRIES

### 3.3.1 Smooth Star-Shaped Domain

#### Problem Definition

Laplace equation in a star-shaped domain ($r(\theta) = 1 + 0.3\cos(5\theta)$):

$$\nabla^2 u = 0 \tag{3.12}$$

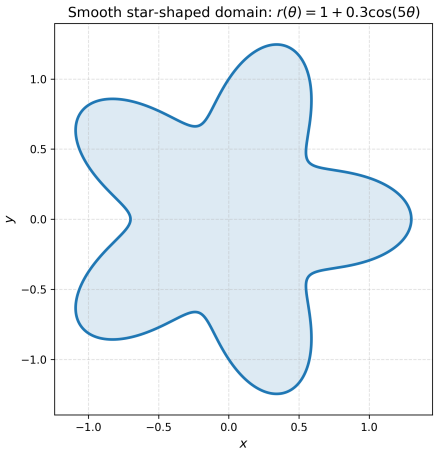



Figure 3.5: Boundary of the smooth star-shaped domain.

**Boundary Condition:**

$$u(x, y) = x^3 - 3xy^2 \tag{3.13}$$

**Analytical Solution:**

$$u(x, y) = x^3 - 3xy^2 \tag{3.14}$$

**Scheme 1: Indirect Double Layer Potential — Collocation — Constant Elements**

**Generative Efficiency:** All models generated valid solvers zero-shot ($I_c = 0$), but their geometric discretization strategies differed. Gemini Pro implemented mathematically consistent straight-line polygonal panels, applying the correct $-0.5$ jump term and bypassing self-integration – valid since the DLP kernel vanishes on flat segments. ChatGPT evaluated uniformly spaced angles but connected them with straight segments, resulting in varying physical element lengths that violate uniform mesh assumptions. Claude utilized the exact curved parametric geometry. However, it improperly applied standard 8-point Gauss quadrature with a distance cutoff for self-elements, failing to account for the non-zero DLP kernel on curved boundaries.

**Computational Performance:** The resulting systems were well-conditioned, with GMRES converging in 8 to 9 iterations across all models. Gemini Pro achieved an $O(h^2)$ superconvergence rate, reaching a terminal relative $L_2$ error of $9.82 \times 10^{-7}$ at $M = 5, 120$. Claude achieved an identical terminal error and convergence rate using exact curvature, despite its simplified self-element integration. ChatGPT terminated at a relative error of $1.28 \times 10^{-6}$ at $M = 5, 120$. For the $M = 5, 120$ mesh, ChatGPT was highly efficient, executing in 1.49 seconds, compared to Claude's 11.46 seconds and Gemini Pro's 21.05 seconds.

Table 3.43: Mesh refinement study of the Gemini Pro Indirect DLP Constant Collocation BEM solver on the Star-shaped domain.

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,280 | 1.572e-05 | 0.013 | 0.045 | 0.004 | 0.062 |
| 2,560 | 5.109e-06 | 0.012 | 0.169 | 0.006 | 0.187 |
| 5,120 | 9.823e-07 | 0.023 | 20.664 | 0.360 | 21.046 |

Table 3.44: Mesh refinement study of the ChatGPT Indirect DLP Constant Collocation BEM solver on the Star-shaped domain.

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,280 | 2.043e-05 | 0.040 | 0.050 | 0.010 | 0.100 |
| 2,560 | 5.109e-06 | 0.090 | 0.270 | 0.010 | 0.380 |
| 5,120 | 1.277e-06 | 0.150 | 1.310 | 0.030 | 1.490 |

Table 3.45: Mesh refinement study of the Claude Indirect DLP Constant Collocation BEM solver on the Star-shaped domain.

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,280 | 1.572e-05 | 0.257 | 0.596 | 0.035 | 0.946 |
| 2,560 | 3.929e-06 | 0.311 | 2.347 | 0.139 | 2.973 |
| 5,120 | 9.823e-07 | 1.038 | 8.946 | 0.269 | 11.459 |

**Scheme 2: Indirect Double Layer Potential — Collocation — Linear Elements**

**Generative Efficiency:** All three models generated valid solvers zero-shot. Gemini Pro and ChatGPT discretized the domain using a straight-line polygonal approximation based on uniform angular sampling. Claude evaluated the exact curved parametric geometry at quadrature points to map the linear basis functions directly onto the true boundary. To address nodal singularities, Gemini Pro applied the equipotential trick to calculate diagonal entries algebraically. ChatGPT collocated at element midpoints to avoid vertex normal ambiguity and applied the appropriate modified jump term. Claude applied the analytical boundary jump term for nodal collocation but relied on standard

Gauss-Legendre quadrature for adjacent elements instead of implementing specialized singular integration.

**Computational Performance:** The implementations yielded well-conditioned systems. The matrix-free GMRES solver converged in 9 iterations for Gemini Pro and Claude, and 11 to 12 iterations for ChatGPT. All solvers exhibited a quadratic convergence rate. At the finest resolution of 5,120 elements, ChatGPT achieved the lowest terminal relative $L_2$ error of $9.56 \times 10^{-7}$. Gemini Pro and Claude yielded comparable errors of $1.91 \times 10^{-6}$ and $1.96 \times 10^{-6}$ respectively. Regarding computational throughput, Gemini Pro executed the 5,120-node system in 1.57 seconds, Claude in 7.30 seconds, and ChatGPT in 18.88 seconds.

Table 3.46: Mesh refinement study of the Gemini Pro Indirect DLP Linear Collocation BEM solver on the Star-shaped domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---|---|---|---|---|
| 1,280 | 3.077e-05 | 0.007 | 0.109 | 0.007 | 0.124 |
| 2,560 | 7.668e-06 | 0.016 | 0.429 | 0.013 | 0.458 |
| 5,120 | 1.914e-06 | 0.032 | 1.512 | 0.023 | 1.566 |

Table 3.47: Mesh refinement study of the ChatGPT Indirect DLP Linear Collocation BEM solver on the Star-shaped domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---|---|---|---|---|
| 1,280 | 1.531e-05 | 0.000 | 1.040 | 0.070 | 1.110 |
| 2,560 | 3.825e-06 | 0.000 | 4.160 | 0.120 | 4.290 |
| 5,120 | 9.560e-07 | 0.000 | 18.630 | 0.240 | 18.880 |

Table 3.48: Mesh refinement study of the Claude Indirect DLP Linear Collocation BEM solver on the Star-shaped domain.

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,280 | 3.143e-05 | 0.125 | 0.486 | 0.034 | 0.687 |
| 2,560 | 7.859e-06 | 0.243 | 1.545 | 0.070 | 2.029 |
| 5,120 | 1.965e-06 | 0.439 | 6.055 | 0.122 | 7.304 |

**Scheme 3: Indirect Double Layer Potential — Galerkin — Linear Elements**

**Generative Efficiency:** All models generated valid solvers zero-shot. ChatGPT discretized the domain using a straight-line polygonal approximation and derived analytical expressions for near-singular adjacent elements. Gemini Pro's initial solver exhibited a linear convergence rate ($O(h)$), yielding a relative $L_2$ error of $6.75 \times 10^{-6}$ in 38.36 seconds at $M = 5,120$. When prompted to improve convergence, Gemini Pro transitioned to evaluating the exact curved geometry and applied the analytical limit of the DLP kernel. This improved the error to $1.39 \times 10^{-10}$ but increased execution time to 955 seconds. A subsequent acceleration prompt yielded the final optimized geometry-caching architecture. Claude's zero-shot implementation relied on 16-point Gauss-Legendre quadrature with a distance cutoff; explicit prompts to improve its convergence order were unsuccessful.

**Computational Performance:** The matrix-free GMRES solvers were well-conditioned across all implementations. ChatGPT and the iteratively prompted Gemini Pro model achieved high-order spatial accuracy, with terminal relative $L_2$ errors of $8.68 \times 10^{-10}$ and $1.39 \times 10^{-10}$ respectively at $M = 5,120$. Claude's reliance on standard numerical quadrature for near-singular integrals restricted its convergence to a quadratic rate, yielding a terminal error of $1.92 \times 10^{-6}$.

Regarding execution time at the finest mesh resolution, Gemini Pro's final accelerated formulation executed in 50.00 seconds. ChatGPT executed its formulation in 100.79

seconds. Claude's double numerical integration over the exact curved geometry required 317.46 seconds.

Table 3.49: Mesh refinement study of the Gemini Pro Indirect DLP Linear Galerkin BEM solver on the Star-shaped domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,280 | 9.099e-10 | 10.132 | 3.016 | 0.018 | 13.166 |
| 2,560 | 1.528e-10 | 0.002 | 14.344 | 0.030 | 14.376 |
| 5,120 | 1.391e-10 | 0.002 | 49.955 | 0.039 | 49.996 |

[a] The setup time for the initial mesh includes Numba JIT compilation overhead.

Table 3.50: Mesh refinement study of the ChatGPT Indirect DLP Linear Galerkin BEM solver on the Star-shaped domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,280 | 5.605e-08 | 0.001 | 6.748 | 0.031 | 6.780 |
| 2,560 | 6.965e-09 | 0.001 | 25.909 | 0.069 | 25.979 |
| 5,120 | 8.682e-10 | 0.001 | 100.653 | 0.133 | 100.787 |

Table 3.51: Mesh refinement study of the Claude Indirect DLP Linear Galerkin BEM solver on the Star-shaped domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,280 | 3.071e-05 | 0.325 | 16.790 | 0.028 | 17.779 |
| 2,560 | 7.678e-06 | 0.584 | 56.414 | 0.103 | 60.904 |
| 5,120 | 1.920e-06 | 2.282 | 302.926 | 0.234 | 317.455 |

### Visual Summary

Figure 3.6 provides a visual summary of the performance for the finest meshes evaluated across all formulations in the star-shaped domain, illustrating the time-to-accuracy trade-offs and capability limits discussed in the preceding sections. The plot is on a log-log scale.

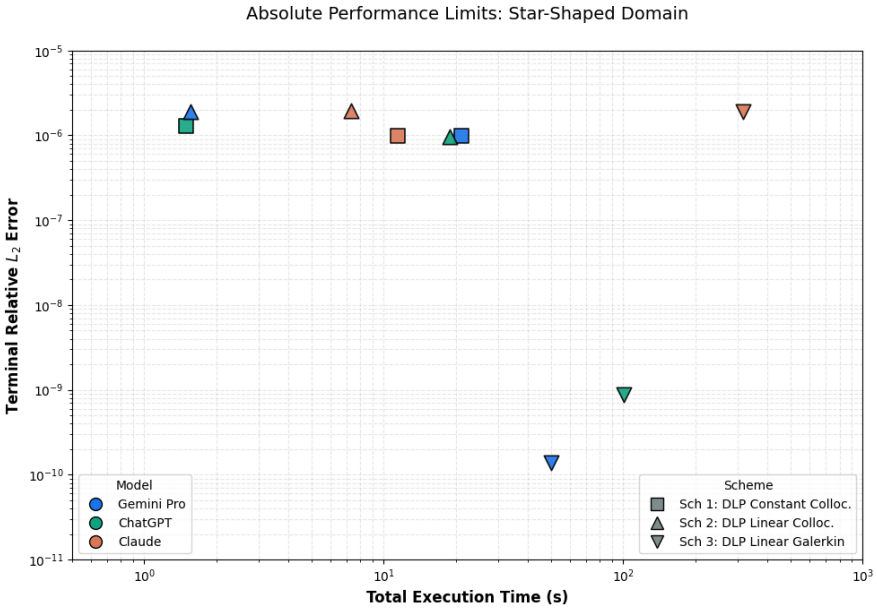



Figure 3.6: $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for Star-shaped domain

### 3.3.2 M-Shaped Domain

#### Problem Definition

Laplace equation with pure Dirichlet BC in a non-convex, M-shaped polygonal domain:

$$\nabla^2 u = 0 \tag{3.15}$$

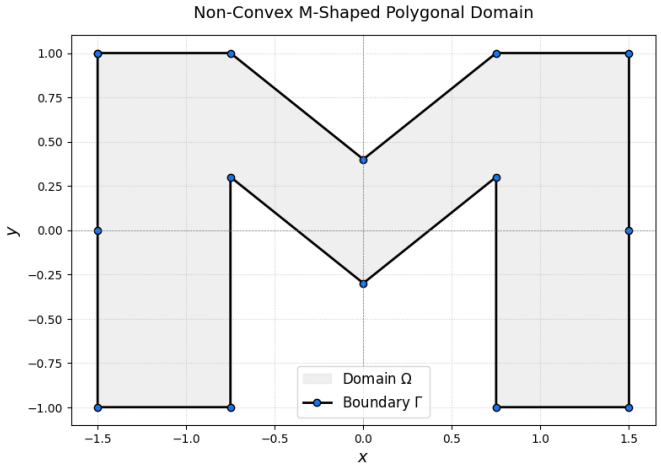



Figure 3.7: Boundary of the non-convex M-shaped domain.

**Boundary Condition:** $u(x, y) = x^3 - 3xy^2$

**Analytical Solution:** $u(x, y) = x^3 - 3xy^2$

**Scheme 1: Indirect Double Layer Potential — Collocation — Constant Elements**

**Generative Efficiency:** All models generated valid solvers zero-shot. While ChatGPT utilized uniform proportional segment scaling for the mesh, Gemini Pro and Claude applied cosine-based mesh grading to resolve flux at the sharp corners.

**Computational Performance:** All solvers achieved an $O(h^2)$ convergence rate, yielding an identical terminal relative $L_2$ error of $1.78 \times 10^{-6}$ at $M = 6,400$. GMRES converged in 20–21 iterations for Gemini Pro, 20–23 iterations for ChatGPT, and 30–33 iterations for Claude's constrained system. At the 6,400-element resolution, total execution times were 8.47 s for ChatGPT, 8.88 s for Gemini Pro, and 16.04 s for Claude.

Table 3.52: Mesh refinement study of the Gemini Pro Indirect DLP Constant Collocation BEM solver on the M-shaped domain.

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,600 | 2.854e-05 | 0.104 | 0.314 | 0.187 | 0.605 |
| 3,200 | 7.133e-06 | 0.170 | 1.431 | 0.351 | 1.953 |
| 6,400 | 1.784e-06 | 0.294 | 7.402 | 1.181 | 8.876 |

Table 3.53: Mesh refinement study of the ChatGPT Indirect DLP Constant Collocation BEM solver on the M-shaped domain.

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,600 | 2.849e-05 | 0.071 | 0.240 | 0.124 | 0.435 |
| 3,200 | 7.131e-06 | 0.152 | 0.916 | 0.237 | 1.305 |
| 6,400 | 1.783e-06 | 0.357 | 7.036 | 1.076 | 8.468 |

Table 3.54: Mesh refinement study of the Claude Indirect DLP Constant Collocation BEM solver on the M-shaped domain.

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,600 | 2.853e-05 | 0.058 | 0.754 | 0.381 | 1.211 |
| 3,200 | 7.135e-06 | 0.283 | 3.937 | 0.747 | 5.107 |
| 6,400 | 1.784e-06 | 0.432 | 13.976 | 1.186 | 16.036 |

**Scheme 2: Indirect Double Layer Potential — Collocation — Linear Elements**

**Generative Efficiency:** Gemini Pro generated a numerically stable solver zero-shot ($I_c = 0$), but it exhibited a suboptimal convergence rate. Upon a prompt requesting improved convergence ($I_c = 1$), Gemini Pro implemented the equipotential trick to compute the singular diagonal entries algebraically ($c_i = -1 - \sum_{j \neq i} K_{ij}$). Claude and ChatGPT required one feedback cycle ($I_c = 1$) to resolve normal vector sign convention errors and achieve numerical stability. Following this, both models also applied the equipotential trick, but subsequent prompts to improve their suboptimal convergence rates were unsuccessful.

**Computational Performance:** Gemini Pro's zero-shot formulation yielded a convergence rate of $O(h^{1.19})$ and a terminal relative $L_2$ error of $3.37 \times 10^{-5}$. The prompted formulation utilizing the equipotential trick achieved the theoretical $O(h^2)$ convergence rate, yielding a terminal error of $2.50 \times 10^{-6}$ at $M = 6,400$ in 14.13 seconds.

Claude and ChatGPT maintained suboptimal convergence rates of approximately $O(h^{1.16})$, resulting in terminal errors of $3.94 \times 10^{-5}$ at $M = 6,400$. At this finest mesh resolution, ChatGPT executed in 5.44 seconds, while Claude required 19.23 seconds. All resulting linear systems were well-conditioned, with GMRES converging in 20–23 iterations across all models.

Table 3.55: Mesh refinement study of the Gemini Pro Indirect DLP Linear Collocation BEM solver on the M-shaped domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---:|---:|---:|---:|---:|
| 1,600 | 4.009e-05 | 0.106 | 0.494 | 0.253 | 0.853 |
| 3,200 | 1.000e-05 | 0.179 | 2.131 | 0.850 | 3.160 |
| 6,400 | 2.501e-06 | 0.525 | 11.732 | 1.873 | 14.130 |

Table 3.56: Mesh refinement study of the ChatGPT Indirect DLP Linear Collocation BEM solver on the M-shaped domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---:|---:|---:|---:|---:|
| 1,600 | 1.940e-04 | 0.090 | 0.460 | 0.230 | 0.780 |
| 3,200 | 8.776e-05 | 0.140 | 1.550 | 0.410 | 2.090 |
| 6,400 | 3.940e-05 | 0.210 | 4.570 | 0.660 | 5.440 |

Table 3.57: Mesh refinement study of the Claude Indirect DLP Linear Collocation BEM solver on the M-shaped domain.

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,600 | 1.932e-04 | 0.180 | 1.360 | 0.540 | 2.070 |
| 3,200 | 8.759e-05 | 0.350 | 3.820 | 1.040 | 5.210 |
| 6,400 | 3.937e-05 | 0.910 | 16.240 | 2.080 | 19.230 |

**Scheme 3: Indirect Double Layer Potential — Galerkin — Linear Elements**

**Generative Efficiency:** Gemini Pro generated a valid solver zero-shot ($I_c = 0$), implementing Duffy transformations for near-singular adjacent integrals, the equipotential trick for system diagonals, and cosine-based mesh grading. ChatGPT also successfully generated a stable solver, correctly applying Jacobian scaling factors during the Duffy coordinate mapping. Claude failed to produce a numerically stable solver after five feedback cycles ($I_c = 5$). Its implementation consistently omitted the necessary Jacobian multiplier during the Duffy transformation, resulting in incorrect matrix assembly.

**Computational Performance:** Gemini Pro and ChatGPT achieved near-cubic convergence rates of 2.95 and 2.88, respectively, reaching double-precision limits at $M = 6,400$. Gemini Pro yielded a terminal relative $L_2$ error of $1.94 \times 10^{-10}$ in 17.87 seconds, and ChatGPT achieved $2.26 \times 10^{-10}$ in 13.04 seconds. Claude's formulation failed to converge, stagnating at a relative $L_2$ error of 5.64.

Table 3.58: Mesh refinement study of the Gemini Pro Indirect DLP Linear Galerkin BEM solver on the M-shaped domain.

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,600 | 1.583e-09 | 0.786 | 0.197 | 0.337 | 1.319 |
| 3,200 | 1.502e-10 | 2.919 | 0.705 | 0.688 | 4.313 |
| 6,400 | 1.938e-10 | 13.247 | 3.169 | 1.454 | 17.869 |

Table 3.59: Mesh refinement study of the ChatGPT Indirect DLP Linear Galerkin BEM solver on the M-shaped domain.

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,600 | 1.530e-09 | 0.471 | 0.186 | 0.106 | 0.763 |
| 3,200 | 1.811e-10 | 1.814 | 0.713 | 0.206 | 2.732 |
| 6,400 | 2.260e-10 | 8.566 | 3.702 | 0.777 | 13.045 |

### Visual Summary

Figure 3.8 provides a visual summary of the performance for the finest meshes evaluated across all formulations in the M-shaped domain, illustrating the time-to-accuracy trade-offs and capability limits discussed in the preceding sections. The plot is on a log-log scale.

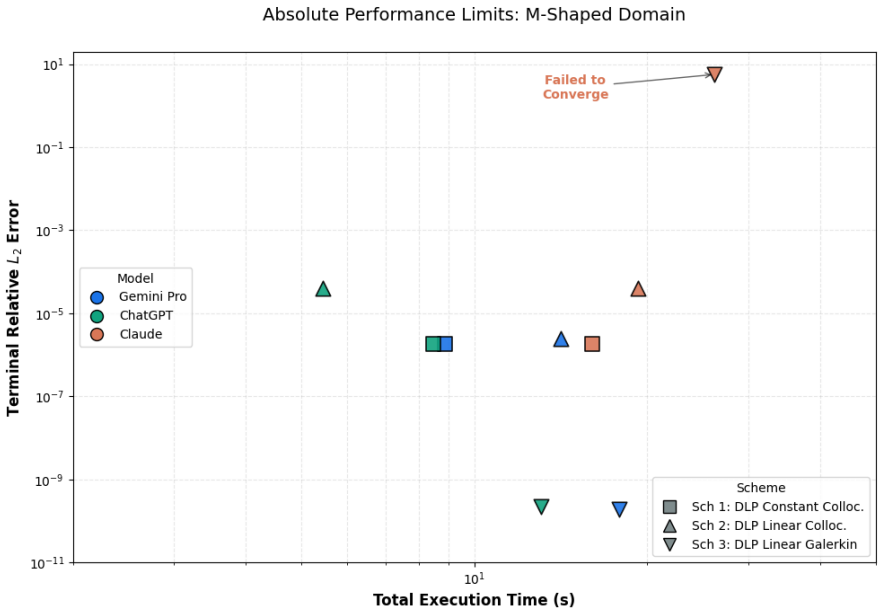



Figure 3.8: $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for M-shaped domain

### 3.3.3 NACA 4318 Airfoil Domain

**Problem Definition**

Laplace equation in the interior of a NACA 4318 airfoil ($m = 0.04$, $p = 0.3$, $t = 0.18$):

$$\nabla^2 u = 0 \tag{3.16}$$

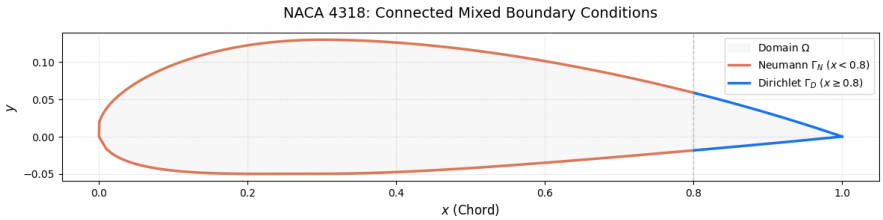



Figure 3.9: NACA 4318 airfoil domain with mixed Dirichlet-Neumann boundary conditions.

**Exact Solution:**

$$u(x, y) = x^3 - 3xy^2 \tag{3.17}$$

**Boundary Conditions:** The boundary $\Gamma$ is split at $x = 0.8$ into Neumann ($\Gamma_N$) and Dirichlet ($\Gamma_D$) segments:

$$q = \nabla u \cdot \mathbf{n} = (3x^2 - 3y^2)n_x - (6xy)n_y \qquad \text{on } \Gamma_N \quad (x < 0.8) \tag{3.18}$$

$$u = x^3 - 3xy^2 \qquad \text{on } \Gamma_D \quad (x \geq 0.8) \tag{3.19}$$

**Scheme 1: Direct Formulation — Collocation — Constant Elements**

**Generative Efficiency:** Gemini Pro and ChatGPT generated valid solvers zero-shot ($I_c = 0$), correctly partitioning the mixed boundary conditions, implementing analytical self-integration for diagonal entries, and applying cosine-based node clustering at the airfoil's edges. Claude failed to produce a converging solver ($I_c = 5$) because it relied exclusively on standard Gaussian quadrature, omitting the necessary near-singular integration treatments for adjacent panels.

**Computational Performance:** Gemini Pro and ChatGPT achieved an $O(h^{2.02})$ convergence rate, yielding identical terminal relative $L_2$ errors of $8.76 \times 10^{-7}$ at $M = 6,400$. At this resolution, ChatGPT executed in 9.53 seconds, and Gemini Pro in 9.66 seconds. Claude's formulation exhibited anti-convergent behavior (order −0.64), with the error increasing to $6.86 \times 10^{-4}$ at $M = 6,400$. Subsequent prompts caused numerical instability, with errors stagnating above 1.14 across all mesh densities.

Table 3.60: Mesh refinement study of the Gemini Pro Direct Constant Collocation BEM solver on the NACA 4318 domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,600 | 1.445e-05 | 0.179 | 0.095 | 0.435 | 0.708 |
| 3,200 | 3.576e-06 | 0.789 | 0.586 | 0.920 | 2.295 |
| 6,400 | 8.760e-07 | 3.418 | 4.528 | 1.710 | 9.656 |

Table 3.61: Mesh refinement study of the ChatGPT Direct Constant Collocation BEM solver on the NACA 4318 domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,600 | 1.445e-05 | 1.674 | 0.091 | 1.437 | 3.202 |
| 3,200 | 3.575e-06 | 0.619 | 0.589 | 0.908 | 2.116 |
| 6,400 | 8.760e-07 | 3.405 | 4.229 | 1.899 | 9.533 |

Table 3.62: Initial mesh refinement study of the Claude Direct Constant Collocation BEM solver on the NACA 4318 domain (Anti-convergent).

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,600 | 5.100e-04 | 0.118 | 0.186 | 0.440 | 0.740 |
| 3,200 | 6.525e-04 | 0.395 | 1.079 | 0.860 | 2.340 |
| 6,400 | 6.862e-04 | 2.163 | 7.156 | 1.870 | 11.190 |

**Scheme 2: Direct Formulation — Collocation — Linear Elements**

**Generative Efficiency:** All models generated stable solvers zero-shot. They implemented analytical integration for singular single-layer kernels and the equipotential trick for

double-layer diagonals. To resolve multi-valued normal fluxes at the trailing edge, Gemini Pro and ChatGPT incorporated double-node topologies. Claude utilized continuous nodal elements, omitting geometric treatment for the trailing edge singularity.

**Computational Performance:** Gemini Pro and ChatGPT achieved an $O(h^2)$ convergence rate. Both yielded terminal relative $L_2$ errors of $1.08 \times 10^{-6}$ at $M = 6,400$. ChatGPT executed in 19.06 seconds and Gemini Pro in 19.43 seconds. Claude exhibited a suboptimal convergence order of 1.65 due to its continuous element formulation, resulting in a terminal error of $3.25 \times 10^{-6}$ and an execution time of 26.31 seconds.

Table 3.63: Mesh refinement study of the Gemini Pro Direct Linear Collocation BEM solver on the NACA 4318 domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,600 | 1.738e-05 | 0.410 | 0.129 | 1.324 | 1.862 |
| 3,200 | 4.307e-06 | 1.642 | 0.969 | 2.488 | 5.099 |
| 6,400 | 1.080e-06 | 7.453 | 6.943 | 5.030 | 19.426 |

Table 3.64: Mesh refinement study of the ChatGPT Direct Linear Collocation BEM solver on the NACA 4318 domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,600 | 1.727e-05 | 0.170 | 0.300 | 0.803 | 1.273 |
| 3,200 | 4.322e-06 | 0.755 | 1.916 | 1.888 | 4.559 |
| 6,400 | 1.079e-06 | 3.513 | 11.608 | 3.942 | 19.063 |

Table 3.65: Mesh refinement study of the Claude Direct Linear Collocation BEM solver on the NACA 4318 domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 1,600 | 1.688e-05 | 3.950 | 0.219 | 0.590 | 4.850 |
| 3,200 | 4.924e-06 | 6.190 | 1.184 | 1.080 | 8.460 |
| 6,400 | 3.248e-06 | 14.910 | 8.648 | 2.750 | 26.310 |

**Scheme 3: Direct Formulation — Galerkin — Linear Elements**

**Generative Efficiency:** All models generated functional solvers zero-shot ($I_c = 0$). Each implemented continuous linear elements with cosine mesh clustering, utilizing analytical evaluation for coincident elements and Duffy transformations for adjacent near-singular pairs. Furthermore, all three models correctly incorporated a double-node topology at the trailing edge to resolve multi-valued normal fluxes.

**Computational Performance:** All models achieved an $O(h^2)$ convergence rate (observed orders 1.98 to 2.03). At $M = 6,400$, ChatGPT yielded a terminal relative $L_2$ error of $2.88 \times 10^{-6}$, Claude achieved $3.01 \times 10^{-6}$, and Gemini Pro yielded $4.70 \times 10^{-6}$. The Galerkin double-integration significantly impacted setup costs. At the finest resolution, Gemini Pro executed in 47.38 seconds, ChatGPT in 116.14 seconds, and Claude required 464.42 seconds.

Table 3.66: Mesh refinement study of the Gemini Pro Direct Linear Galerkin BEM solver on the NACA 4318 domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---:|---:|---:|---:|---:|
| 1,600 | 8.133e-05 | 2.490 | 0.126 | 0.487 | 3.102 |
| 3,200 | 1.999e-05 | 9.345 | 0.809 | 0.894 | 11.049 |
| 6,400 | 4.698e-06 | 39.238 | 6.543 | 1.601 | 47.381 |

Table 3.67: Mesh refinement study of the ChatGPT Direct Linear Galerkin BEM solver on the NACA 4318 domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---:|---:|---:|---:|---:|
| 1,600 | 4.743e-05 | 6.520 | 0.262 | 0.703 | 7.485 |
| 3,200 | 1.176e-05 | 26.486 | 1.756 | 1.716 | 29.958 |
| 6,400 | 2.884e-06 | 104.860 | 8.698 | 2.587 | 116.145 |

Table 3.68: Mesh refinement study of the Claude Direct Linear Galerkin BEM solver on the NACA 4318 domain.

| Elements ($M$) | Rel $L_2$ Error | Setup (s) | Solve (s) | Eval (s) | Total (s) |
|---|---|---|---|---|---|
| 1,600 | 4.689e-05 | 40.030 | 0.248 | 0.390 | 40.670 |
| 3,200 | 1.193e-05 | 166.610 | 1.499 | 0.790 | 168.900 |
| 6,400 | 3.013e-06 | 454.240 | 8.164 | 2.000 | 464.420 |

### Visual Summary

Figure 3.10 provides a visual summary of the performance for the finest meshes evaluated across all formulations in the NACA 4318 domain, illustrating the time-to-accuracy trade-offs and capability limits discussed in the preceding sections. The plot is on a log-log scale.




Figure 3.10: $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for NACA 4318 Airfoil

## 3.4 CROSS-DOMAIN KNOWLEDGE TRANSFER

This section evaluates in-context knowledge transfer capabilities. In prior evaluations, all three models failed to produce numerically stable solvers for the rectangular domain

across all direct formulation schemes, exceeding our empirical maximum feedback threshold. To evaluate cross-geometry adaptation, the models were tested using a one-shot prompting strategy. They were provided with the successful annulus domain solvers—previously generated by Gemini for mixed boundary conditions—as in-context references. This approach tests whether the models can abstract the core integration treatments from a curved geometry and successfully apply them to the sharp-cornered rectangular domain.

---

**System Prompt: Cross-Domain BEM Solver Generation**

---

**Context:** Reference 2D BEM Python solver provided for *[Source Domain]*.

**Task:** Generate a new solver for *[Target Domain]* utilizing the exact numerical methodology of the reference code.

**Constraints:**

1. Preserve formulation perfectly (element order, singular integrations, matrix assembly).
2. Modify only: geometry, boundary conditions, exact solution, and evaluation points.
3. Output a single, runnable Python script.

**New Problem Definition:**

- **Equation:** *[Target Equation]*
- **Domain:** *[Target Geometry Definition]*
- **Boundary Conditions:** *[Target Boundary Conditions]*
- **Exact Solution:** *[Target Exact Solution]*

**Execution Requirements:**

Run a mesh refinement study ($N$ = *[Resolutions]*).

Print a performance table – DOFs, Setup, Solve, Eval times, Rel. $L_2$ Error

**Reference Code (*[Source Domain]*):**

```
[Insert Reference Source Code]
```

---

Figure 3.11: One-shot prompt template utilized for cross-domain knowledge transfer.

**Rectangular Domain (One-Shot Evaluation)**

**Discretization Strategy:** Across the one-shot evaluations for the rectangular domain, the models adopted differing geometric meshing approaches. Gemini Pro and Claude applied aspect-ratio-aware meshing, assigning $2N$ panels to the length and $N$ panels to the width, resulting in $M = 6N$ total elements. ChatGPT applied uniform meshing across all edges regardless of geometric proportion, resulting in $M = 4N$ total elements.

**Scheme 1: Direct Formulation — Collocation — Constant Elements**

**Generative Efficiency:** Using the one-shot annulus reference, all models generated stable solvers. They successfully adapted singular integrations and GMRES assembly to the rectangular domain, resolving their previous zero-shot failures.

**Computational Performance:** All models exhibited a bounded convergence order of 1.63, a known limitation for constant collocation on sharp 90° corners. At their finest resolutions, Gemini Pro achieved $6.71 \times 10^{-6}$ ($M = 3,840$) in 231.15 seconds. ChatGPT yielded $2.70 \times 10^{-5}$ ($M = 2,560$) in 124.60 seconds. Claude yielded a relative $L_2$ error of $6.71 \times 10^{-6}$ ($M = 3,840$) in 244.20 seconds.

Table 3.69: Mesh refinement study of the Gemini Pro Direct Constant Collocation BEM solver on the Rectangular domain (One-Shot).

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---|---|---|---|---|
| 960 | 6.370e-05 | 0.028 | 8.221 | 0.084 | 8.333 |
| 1,920 | 2.078e-05 | 0.101 | 40.306 | 0.124 | 40.531 |
| 3,840 | 6.708e-06 | 0.298 | 230.538 | 0.312 | 231.147 |

Table 3.70: Mesh refinement study of the ChatGPT Direct Constant Collocation BEM solver on the Rectangular domain (One-Shot).

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|:---:|:---:|:---:|:---:|:---:|
| 640 | 2.687e-04 | 0.016 | 3.244 | 0.046 | 3.305 |
| 1,280 | 8.458e-05 | 0.034 | 18.001 | 0.093 | 18.129 |
| 2,560 | 2.703e-05 | 0.190 | 124.243 | 0.168 | 124.600 |

Table 3.71: Mesh refinement study of the Claude Direct Constant Collocation BEM solver on the Rectangular domain (One-Shot).

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|:---:|:---:|:---:|:---:|:---:|
| 960 | 6.370e-05 | 0.016 | 4.017 | 0.053 | 4.086 |
| 1,920 | 2.078e-05 | 0.073 | 27.305 | 0.139 | 27.517 |
| 3,840 | 6.709e-06 | 0.387 | 243.561 | 0.255 | 244.203 |

**Scheme 2: Direct Formulation — Collocation — Linear Elements**

**Generative Efficiency:** Using the one-shot annulus reference, all models generated stable solvers. They successfully adapted continuous linear elements, singular integrations, nodal matrix assembly, and corner $c$-coefficient treatments to the rectangular domain.

**Computational Performance:** All models exhibited convergence orders between 1.83 and 1.95, approaching the theoretically optimal $O(h^2)$ rate for continuous linear elements. At their finest resolutions, Gemini Pro achieved $1.18 \times 10^{-5}$ ($M = 1,920$) in 86.83 seconds. ChatGPT yielded $1.66 \times 10^{-5}$ ($M = 2,560$) in 249.54 seconds. Claude yielded a relative $L_2$ error of $1.18 \times 10^{-5}$ ($M = 1,920$) in 78.98 seconds.

Table 3.72: Mesh refinement study of the Gemini Pro Direct Linear Collocation BEM solver on the Rectangular domain (One-Shot).

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 480 | 1.546e-04 | 0.008 | 1.163 | 0.032 | 1.203 |
| 960 | 4.427e-05 | 0.036 | 10.007 | 0.083 | 10.126 |
| 1,920 | 1.184e-05 | 0.117 | 86.177 | 0.532 | 86.826 |

Table 3.73: Mesh refinement study of the ChatGPT Direct Linear Collocation BEM solver on the Rectangular domain (One-Shot).

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 640 | 2.492e-04 | 0.009 | 4.971 | 0.040 | 5.020 |
| 1,280 | 6.389e-05 | 0.038 | 37.109 | 0.095 | 37.242 |
| 2,560 | 1.663e-05 | 0.230 | 249.106 | 0.201 | 249.537 |

Table 3.74: Mesh refinement study of the Claude Direct Linear Collocation BEM solver on the Rectangular domain (One-Shot).

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 480 | 1.546e-04 | 0.021 | 2.795 | 0.067 | 2.884 |
| 960 | 4.427e-05 | 0.047 | 16.672 | 0.101 | 16.821 |
| 1,920 | 1.184e-05 | 0.138 | 78.703 | 0.142 | 78.983 |

**Scheme 3: Direct Formulation — Galerkin — Linear Elements**

**Generative Efficiency:** Using the one-shot annulus reference, all models successfully generated stable Galerkin BEM solvers. They accurately handled the double integration required for continuous linear elements and successfully adapted the semi-analytical singular treatments to the rectangular domain without issue.

**Computational Performance:** The models exhibited varying convergence orders between 1.41 and 2.03, demonstrating the higher accuracy and stability characteristic of Galerkin integration on non-smooth domains. At their finest resolutions, Gemini Pro achieved $8.43 \times 10^{-6}$ ($M = 1,920$) in 11.47 seconds. ChatGPT yielded $3.10 \times 10^{-5}$

($M = 2,560$) in 18.81 seconds. Claude yielded a relative $L_2$ error of $7.66 \times 10^{-6}$ ($M = 1,920$) in 6.66 seconds.

Table 3.75: Mesh refinement study of the Gemini Pro Direct Linear Galerkin BEM solver on the Rectangular domain (One-Shot).

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 480 | 1.463e-04 | 0.423 | 0.560 | 0.080 | 1.062 |
| 960 | 3.782e-05 | 1.867 | 1.619 | 0.202 | 3.688 |
| 1,920 | 8.434e-06 | 6.482 | 4.620 | 0.370 | 11.472 |

Table 3.76: Mesh refinement study of the ChatGPT Direct Linear Galerkin BEM solver on the Rectangular domain (One-Shot).

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 640 | 1.698e-04 | 0.553 | 0.481 | 0.068 | 1.103 |
| 1,280 | 6.740e-05 | 1.870 | 1.475 | 0.181 | 3.526 |
| 2,560 | 3.103e-05 | 9.676 | 8.490 | 0.649 | 18.814 |

Table 3.77: Mesh refinement study of the Claude Direct Linear Galerkin BEM solver on the Rectangular domain (One-Shot).

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 480 | 1.275e-04 | 0.220 | 0.190 | 0.057 | 0.467 |
| 960 | 3.223e-05 | 0.977 | 0.415 | 0.118 | 1.510 |
| 1,920 | 7.660e-06 | 3.750 | 2.701 | 0.213 | 6.664 |

### Visual Summary

Figure 3.12 provides a visual summary of the performance for the finest meshes evaluated across all formulations in the rectangular domain with a one-shot reference of the annular domain, illustrating the time-to-accuracy trade-offs and capability limits discussed in the preceding sections. The plot is on a log-log scale.

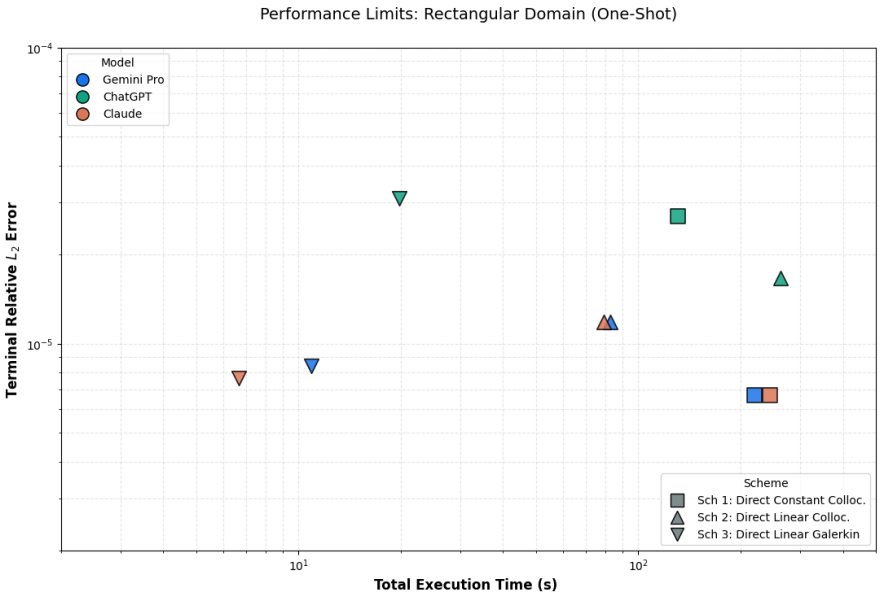



Figure 3.12: $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for Rectangle under one-shot prompting

# CHAPTER 4

# LLM CODE SYNTHESIS FOR 3D DOMAINS

## 4.1 INTRODUCTION TO 3D TESTING

This chapter presents the results of LLM-generated BEM solvers applied to three-dimensional (3D) geometries. The results are evaluated for two foundation models: **Gemini Pro** and **ChatGPT (Thinking)**.

**Claude Sonnet** is excluded from 3D testing because preliminary prompts consistently generated numerically unstable formulations, and the iterative debugging process exhausted session limits without producing a functional solver.

## 4.2 NUMERICAL RESULTS FOR 3D DOMAINS

### 4.2.1 Sphere

**Problem Definition: 3D Unit Sphere**

Laplace equation in the spherical domain ($R = 1$):

$$\nabla^2 u = 0 \tag{4.1}$$

**Boundary Condition (Dirichlet):**

$$u(x, y, z)|_\Gamma = x^2 - y^2 \tag{4.2}$$

**Analytical Solution:**

$$u(x, y, z) = x^2 - y^2 \tag{4.3}$$

**Scheme 1: Indirect Formulation — Double Layer Potential — Constant Collocation**

**Generative Efficiency:** Both models generated stable solvers in the first attempt for the requested spherical domain. Gemini Pro implemented a 7-point Gaussian quadrature for

surface integrals, whereas ChatGPT utilized a 3-point symmetric quadrature.

**Computational Performance:** Gemini Pro achieved a convergence order of 1.54, reaching a relative $L_2$ error of $4.24 \times 10^{-4}$ at $M = 20,480$ in 18.88 seconds. ChatGPT recorded a convergence order of 0.61, yielding a terminal error of $1.17 \times 10^{-3}$ at $M = 20,480$ in 8.62 seconds.

Table 4.1: Mesh refinement study of the Gemini Pro Indirect DLP Constant Collocation BEM solver on the Sphere domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---|---|---|---|---|
| 1,280 | 3.604e-03 | 0.012 | 1.077 | 0.812 | 1.901 |
| 5,120 | 1.160e-03 | 0.039 | 0.963 | 0.107 | 1.109 |
| 20,480 | 4.240e-04 | 0.144 | 18.091 | 0.641 | 18.876 |

Table 4.2: Mesh refinement study of the ChatGPT Indirect DLP Constant Collocation BEM solver on the Sphere domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---|---|---|---|---|
| 1,280 | 2.702e-03 | 0.142 | 0.848 | 0.695 | 1.685 |
| 5,120 | 2.023e-03 | 0.580 | 0.366 | 0.048 | 0.994 |
| 20,480 | 1.167e-03 | 2.182 | 6.210 | 0.229 | 8.621 |

**Scheme 2: Indirect Formulation — Collocation — Linear Elements**

**Generative Efficiency:** Both models generated stable solvers in the first attempt for the requested spherical domain. Both successfully implemented the continuous linear collocation methodology, utilizing a 7-point quadrature scheme to evaluate the surface integrals over the elements.

**Computational Performance:** Both models achieved an observed convergence order of 2.03. At $N = 10,242$, Gemini Pro and ChatGPT yielded an identical relative $L_2$ error of $4.58 \times 10^{-4}$. Gemini Pro recorded a total execution time of 13.32 seconds, while ChatGPT executed in 19.66 seconds.

Table 4.3: Mesh refinement study of the Gemini Pro Indirect Collocation Linear BEM solver on the Sphere domain.

| Elements ($M$) | Rel $L_2$ Error | Setup (s) | Solve (s) | Eval (s) | Total (s) |
|---|---|---|---|---|---|
| 642 | 7.622e-03 | 0.988 | 1.124 | 0.946 | 3.058 |
| 2,562 | 1.861e-03 | 0.119 | 0.569 | 0.442 | 1.130 |
| 10,242 | 4.585e-04 | 1.432 | 9.799 | 2.094 | 13.324 |

[a] The setup time for the initial mesh includes Numba JIT compilation overhead.

Table 4.4: Mesh refinement study of the ChatGPT Indirect Collocation Linear BEM solver on the Sphere domain.

| Elements ($M$) | Rel $L_2$ Error | Setup (s) | Solve (s) | Eval (s) | Total (s) |
|---|---|---|---|---|---|
| 642 | 7.622e-03 | 0.205 | 0.046 | 0.169 | 0.420 |
| 2,562 | 1.861e-03 | 0.780 | 0.665 | 0.612 | 2.056 |
| 10,242 | 4.585e-04 | 4.023 | 12.751 | 2.889 | 19.664 |

**Scheme 3: Indirect — Double Layer Potential — Galerkin — Linear Elements**

**Generative Efficiency:** Both models generated stable solvers in the first attempt for the requested spherical domain.

**Computational Performance:** Gemini Pro achieved a convergence order of 1.08, reaching a relative $L_2$ error of $2.46 \times 10^{-4}$ at $N = 10,242$ in 164.85 seconds. ChatGPT recorded a convergence order of 1.05, yielding an error of $4.91 \times 10^{-4}$ at $N = 10,242$ in 107.03 seconds.

Table 4.5: Mesh refinement study of the Gemini Pro Indirect DLP Galerkin Linear BEM solver on the Sphere domain.

| Elements ($M$) | Rel $L_2$ Error | Setup (s) | Solve (s) | Eval (s) | Total (s) |
|---|---|---|---|---|---|
| 642 | 1.099e-03 | 4.043 | 0.007 | 1.823 | 5.873 |
| 2,562 | 5.047e-04 | 14.269 | 0.051 | 1.215 | 15.535 |
| 10,242 | 2.464e-04 | 160.536 | 0.588 | 3.723 | 164.847 |

Table 4.6: Mesh refinement study of the ChatGPT Indirect DLP Galerkin Linear BEM solver on the Sphere domain.

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---:|---:|---:|---:|---:|
| 642 | 2.101e-03 | 0.321 | 0.007 | 0.243 | 0.571 |
| 2,562 | 9.979e-04 | 6.115 | 0.048 | 0.796 | 6.959 |
| 10,242 | 4.913e-04 | 103.237 | 0.668 | 3.093 | 107.031 |

### Visual Summary

Figure 4.1 provides a visual summary of the performance for the finest meshes evaluated across all formulations in the spherical domain, illustrating the time-to-accuracy trade-offs and capability limits discussed in the preceding sections. The plot is on a log-log scale.

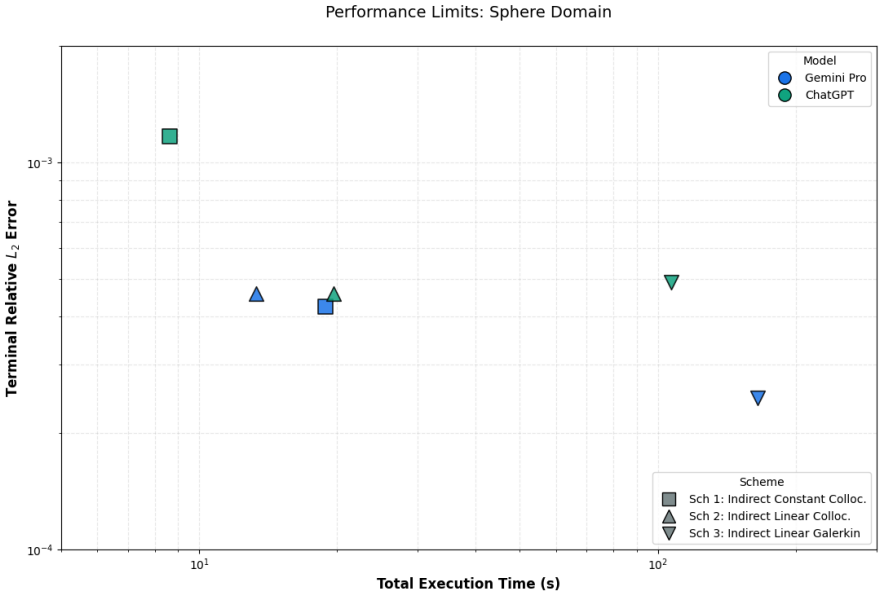



Figure 4.1: $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for the sphere domain

### 4.2.2 Cube

### Problem Definition

Laplace equation in the cubic domain $\Omega = [0, 1]^3$:

$$\nabla^2 u = 0 \tag{4.4}$$

**Boundary Conditions (Mixed):**

**Dirichlet Boundary ($\Gamma_D$)** **Neumann Boundary ($\Gamma_N$)**

$$u = yz \quad \text{on } x = 0 \qquad q = y + z \quad \text{on } x = 1$$

$$u = zx \quad \text{on } y = 0 \qquad q = x + z \quad \text{on } y = 1$$

$$u = xy \quad \text{on } z = 0 \qquad q = x + y \quad \text{on } z = 1$$

**Analytical Solution:**

$$u(x, y, z) = xy + yz + zx \tag{4.5}$$

**Scheme 1: Direct Formulation — Collocation — Constant Elements**

**Generative Efficiency:** Gemini Pro generated a stable solver in the first attempt for the cubical domain. ChatGPT required a second attempt. In its initial generation, ChatGPT applied a sign reversal in the interior evaluation boundary integral, resulting in divergent interior potentials. This error was identified and resolved at $I_c = 1$.

**Computational Performance:** Gemini Pro achieved an observed convergence order of 1.80, reaching a relative $L_2$ error of $6.502 \times 10^{-4}$ at $M = 12,288$ with a total execution time of 43.178 seconds. ChatGPT recorded a convergence order of 1.75, yielding a relative $L_2$ error of $9.271 \times 10^{-4}$ at $M = 12,288$ with a total execution time of 24.349 seconds.

Table 4.7: Mesh refinement study of the Gemini Pro Direct Collocation Constant BEM solver on the Cube domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|:---:|:---:|:---:|:---:|:---:|
| 768 | 7.868e-03 | 0.061 | 0.097 | 0.007 | 0.165 |
| 3,072 | 2.327e-03 | 0.625 | 0.436 | 0.035 | 1.096 |
| 12,288 | 6.502e-04 | 14.018 | 28.941 | 0.219 | 43.178 |

Table 4.8: Mesh refinement study of the ChatGPT Direct Collocation Constant BEM solver on the Cube domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 768 | 1.044e-02 | 0.061 | 0.014 | 0.001 | 0.076 |
| 3,072 | 3.156e-03 | 0.592 | 1.164 | 0.010 | 1.766 |
| 12,288 | 9.271e-04 | 8.642 | 15.673 | 0.034 | 24.349 |

**Scheme 2: Direct Formulation — Collocation — Linear Elements**

**Generative Efficiency:** Gemini ($I_c = 0$) implemented the methodology in the first attempt, applying Duffy transformations for singular integration and assembling the dense system. ChatGPT ($I_c = 5$) matched the structural requirements of the scheme but retained integration inaccuracies after five feedback iterations.

**Computational Performance:** Gemini achieved a convergence order of 1.90, approaching the theoretical $O(h^2)$ rate. At $M = 12,288$, Gemini yielded a relative $L_2$ error of $4.22 \times 10^{-4}$, executing in 7.84 seconds. ChatGPT exhibited a convergence order of 1.01. At $M = 12,288$, ChatGPT produced a relative $L_2$ error of $1.48 \times 10^{-2}$, executing in 5.82 seconds.

Table 4.9: Mesh refinement study of the Gemini Direct Linear Collocation BEM solver on the Cube domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---|---|---|---|---|---|
| 768 | 5.912e-03 | 0.016 | 0.121 | 0.011 | 0.148 |
| 3,072 | 1.603e-03 | 0.207 | 0.100 | 0.057 | 0.364 |
| 12,288 | 4.220e-04 | 3.030 | 4.543 | 0.266 | 7.838 |

Table 4.10: Mesh refinement study of the ChatGPT Direct Linear Collocation BEM solver on the Cube domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---:|---:|---:|---:|---:|
| 768 | 5.961e-02 | 8.358 | 0.013 | 1.307 | 9.678 |
| 3,072 | 2.973e-02 | 0.117 | 0.126 | 0.005 | 0.248 |
| 12,288 | 1.477e-02 | 1.725 | 4.074 | 0.022 | 5.821 |

**Scheme 3: Direct Formulation — Galerkin — Linear Elements**

**Generative Efficiency:** Gemini Pro successfully generated a stable Direct Galerkin solver for the 3D cubic domain ($I_c = 1$). While the initial implementation achieved a relative $L_2$ error of $3.01 \times 10^{-3}$ and a convergence rate of 1.60, subsequent iterative prompting for optimization improved these metrics. ChatGPT failed to produce a stable or convergent solver after five iterations, consistently yielding non-physical solutions with relative errors exceeding 50%.

**Computational Performance:** The refined Gemini Pro solver attained a terminal relative $L_2$ error of $2.25 \times 10^{-3}$ at $M = 12,288$ elements, with the observed convergence rate improving from 1.60 to 1.74. In contrast, ChatGPT exhibited a near-zero convergence slope of 0.16.

Table 4.11: Mesh refinement study of the Gemini Pro Direct Linear Galerkin BEM solver on the 3D Cube domain (Refined).

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---:|---:|---:|---:|---:|
| 768 | 2.507e-02 | 0.360 | 0.006 | 0.002 | 0.367 |
| 3,072 | 7.627e-03 | 2.037 | 0.144 | 0.005 | 2.186 |
| 12,288 | 2.250e-03 | 19.714 | 7.915 | 0.012 | 27.642 |

Table 4.12: Mesh refinement study of the ChatGPT Direct Linear Galerkin BEM solver on the 3D Cube domain (Failed Iteration).

| Elements ($M$) | Rel $L_2$ Error | Setup (s) | Solve (s) | Eval (s) | Total (s) |
|---|---|---|---|---|---|
| 768 | 7.295e-01 | 37.622 | 0.065 | 7.386 | 45.073 |
| 3,072 | 6.600e-01 | 1.038 | 0.794 | 0.020 | 1.851 |
| 12,288 | 5.873e-01 | 6.181 | 22.060 | 0.039 | 28.280 |

**Visual Summary**

Figure 4.2 provides a visual summary of the performance for the finest meshes evaluated across all formulations in the cubic domain, illustrating the time-to-accuracy trade-offs and capability limits discussed in the preceding sections. The plot is on a log-log scale.




Figure 4.2: $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for the cube

### 4.2.3 Skewed parallelepiped

**Problem Definition**

Laplace equation in the non-orthogonal hexahedral domain:

$$\nabla^2 u = 0 \tag{4.6}$$

**Geometric Mapping:** The domain is generated by applying an isoparametric shear mapping to a reference unit cube $(x, y, z) \in [0, 1]^3$:

$$X = x + 0.5y \tag{4.7}$$

$$Y = y + 0.5z \tag{4.8}$$

$$Z = z \tag{4.9}$$

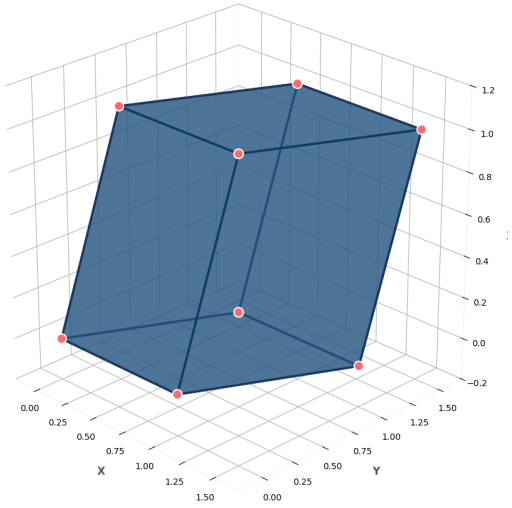



Figure 4.3: Skewed Parallelepiped domain

**Analytical Solution:**

$$u(X, Y, Z) = \sinh(\sqrt{2}X)\sin(Y)\cos(Z) \tag{4.10}$$

**Boundary Conditions (Pure Dirichlet):**

$$u_i = u(X_i, Y_i, Z_i) \quad \text{for } i \in \Gamma \tag{4.11}$$

A point to note is that the solvers utilize an isoparametric mapping approach. A structured reference mesh is first generated on a unit cube ($[0, 1]^3$), and a linear shear transformation is then applied to the nodal coordinates to form the skewed parallelepiped.

**Scheme 1: Indirect — Double Layer Potential — Collocation — Constant Elements**

**Generative Efficiency:** Both models required a second attempt ($I_c = 1$). Initial generations yielded divergent results due to an incorrect +0.5 diagonal jump coefficient for the Double Layer Potential, which was successfully corrected to $-0.5$ to yield stable solvers.

**Computational Performance:** Both models generated mathematically identical solvers, achieving an observed convergence order of 1.95. At $M = 12,288$, both models yielded an identical relative $L_2$ error of $3.654 \times 10^{-5}$. Gemini Pro recorded a total execution time of 24.465 seconds, while ChatGPT executed in 26.445 seconds.

Table 4.13: Mesh refinement study of the Gemini Pro Indirect Constant Collocation BEM solver on the Skewed Parallelepiped domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|:---:|:---:|:---:|:---:|:---:|
| 768 | 5.488e-04 | 0.135 | 0.031 | 0.001 | 0.167 |
| 3,072 | 1.438e-04 | 0.538 | 0.903 | 0.004 | 1.445 |
| 12,288 | 3.654e-05 | 3.475 | 20.937 | 0.053 | 24.465 |

Table 4.14: Mesh refinement study of the ChatGPT Direct Constant Collocation BEM solver on the Skewed Parallelepiped domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|:---:|:---:|:---:|:---:|:---:|
| 768 | 5.488e-04 | 0.022 | 0.014 | 0.001 | 0.037 |
| 3,072 | 1.438e-04 | 0.147 | 0.707 | 0.003 | 0.857 |
| 12,288 | 3.654e-05 | 1.965 | 24.450 | 0.030 | 26.445 |

**Scheme 2: Indirect — Double Layer Potential — Collocation — Linear Elements**

**Generative Efficiency:** Both models generated stable solvers in the first attempt. Gemini Pro successfully handled singular integrations using a 2D Duffy transformation (3x3 Gauss-Legendre). ChatGPT calculated exact analytical solid angles for the diagonal jump coefficient but bypassed near-singular integration on adjacent elements, reducing its numerical accuracy. Both utilized a 7-point Dunavant quadrature for regular elements.

**Computational Performance:** Gemini Pro achieved an observed convergence order of 2.04, reaching a relative $L_2$ error of $1.173 \times 10^{-4}$ at $M = 12,288$ with a total execution time of 8.813 seconds. ChatGPT recorded a convergence order of 1.78, yielding an error of $5.009 \times 10^{-3}$ at $M = 12,288$ with a total execution time of 7.662 seconds.

Table 4.15: Mesh refinement study of the Gemini Pro Indirect DLP Linear BEM solver on the Skewed Parallelepiped domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|:---:|:---:|:---:|:---:|:---:|
| 768 | 1.974e-03 | 0.128 | 0.005 | 0.002 | 0.135 |
| 3,072 | 4.761e-04 | 0.474 | 0.149 | 0.006 | 0.629 |
| 12,288 | 1.173e-04 | 2.471 | 6.325 | 0.017 | 8.813 |

Table 4.16: Mesh refinement study of the ChatGPT Indirect DLP Linear BEM solver on the Skewed Parallelepiped domain.

| **Elements ($N_e$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|:---:|:---:|:---:|:---:|:---:|
| 768 | 5.886e-02 | 0.024 | 0.004 | 0.001 | 0.029 |
| 3,072 | 1.624e-02 | 0.117 | 0.092 | 0.006 | 0.215 |
| 12,288 | 5.009e-03 | 0.774 | 6.867 | 0.021 | 7.662 |

**Scheme 3: Indirect — Double Layer Potential — Galerkin — Linear Elements**

**Generative Efficiency:** ChatGPT generated a stable solver in the first attempt. Gemini Pro required one refinement cycle ($I_c = 1$) to correct an erroneous +0.5 diagonal jump coefficient to the theoretically required $-0.5$. Both models successfully implemented the true Galerkin double integration, utilizing a 7-point Dunavant quadrature for off-diagonal terms and exact mass matrix row sums for singular diagonals.

**Computational Performance:** Both models produced mathematically identical solvers, achieving an optimal $O(h^2)$ convergence order of 2.035. At $M = 12,288$, both models yielded a relative $L_2$ error of $3.444 \times 10^{-4}$. While numerical accuracy was identical, Gemini Pro executed the computationally intensive Galerkin double integrals significantly faster, completing the system assembly in 21.601 seconds compared to 54.050 seconds

for ChatGPT.

Table 4.17: Mesh refinement study of the Gemini Pro Indirect DLP Galerkin Linear BEM solver on the Skewed Parallelepiped domain.

| **Elements ($N_e$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---:|---:|---:|---:|---:|
| 768 | 5.782e-03 | 0.142 | 0.015 | 0.001 | 0.158 |
| 3,072 | 1.400e-03 | 1.207 | 0.155 | 0.004 | 1.366 |
| 12,288 | 3.444e-04 | 21.601 | 4.862 | 0.020 | 26.483 |

Table 4.18: Mesh refinement study of the ChatGPT Indirect DLP Galerkin Linear BEM solver on the Skewed Parallelepiped domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---:|---:|---:|---:|---:|
| 768 | 5.782e-03 | 0.257 | 0.005 | 0.001 | 0.263 |
| 3,072 | 1.400e-03 | 2.508 | 0.108 | 0.003 | 2.619 |
| 12,288 | 3.444e-04 | 54.050 | 5.825 | 0.016 | 59.891 |

**Visual Summary**

Figure 4.4 provides a visual summary of the performance for the finest meshes evaluated across all formulations in the skewed parallelepiped domain, illustrating the time-to-accuracy trade-offs and capability limits discussed in the preceding sections. The plot is on a log-log scale.




Figure 4.4: $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for the skewed parallelepiped domain

### 4.2.4 Perturbed Sphere

#### Problem Definition

Laplace equation in the arbitrary curvilinear domain:

$$\nabla^2 u = 0 \tag{4.12}$$

**Geometric Mapping (Perturbed Cubed-Sphere):** The surface is generated by projecting a cube onto a unit sphere and applying the following radial perturbation:

$$r(\theta, \phi) = 1.5 + 0.3 \sin(4\theta) \cos(5\phi) \tag{4.13}$$

$$X = r \sin(\theta) \cos(\phi) \tag{4.14}$$

$$Y = r \sin(\theta) \sin(\phi) \tag{4.15}$$

$$Z = r \cos(\theta) \tag{4.16}$$

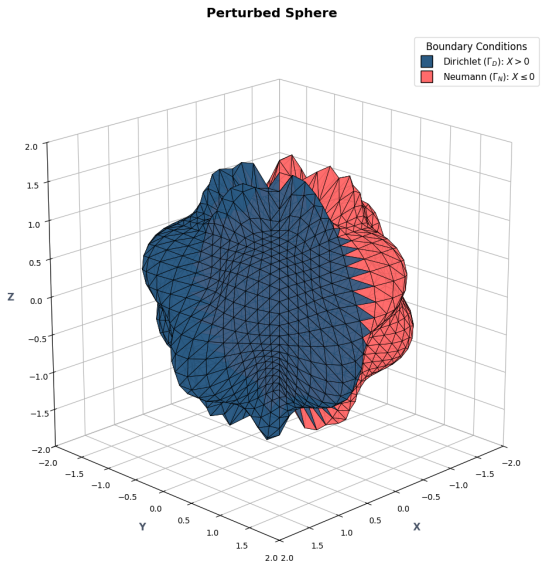



Figure 4.5: Perturbed Sphere domain

**Analytical Solution:**

$$u(X, Y, Z) = \sinh(X)\sin(Y) + \cosh(Y)\cos(Z) \tag{4.17}$$

**Boundary Conditions (Mixed):**

| **Dirichlet Boundary** ($\Gamma_D$) | **Neumann Boundary** ($\Gamma_N$) |
|---|---|
| $u = u_{\text{exact}}$ for $X > 0$ | $q = \nabla u \cdot \hat{n}$ for $X \leq 0$ |

**Scheme 1: Direct Formulation — Collocation — Constant Elements**

**Generative Efficiency:** ChatGPT generated a stable solver in the first attempt ($I_c = 0$). It correctly anticipated the high local curvature of the perturbed sphere, immediately implementing a 144-point mapped Gauss-Legendre quadrature to resolve near-singular adjacent elements alongside a 7-point Dunavant rule for the far-field. Gemini Pro required five refinement cycles ($I_c = 5$). Early iterations exhibited severe anti-convergence due to standard quadrature failing on high-curvature adjacent panels. Stability was restored at $I_c = 5$ by adopting a distance-based near-singular integration scheme identical to

ChatGPT's. To prevent memory overflow during the $12,288 \times 12,288$ dense matrix assembly, both models autonomously deployed Out-Of-Core (OOC) streaming via memory-mapped files coupled with GMRES.

**Computational Performance:** Both models achieved the theoretical $O(h)$ convergence expectation for constant elements on a highly perturbed domain. Gemini Pro recorded a convergence order of 0.741, reaching a relative $L_2$ error of $1.654 \times 10^{-3}$ at $M = 12,288$ in 20.219 seconds. ChatGPT recorded a convergence order of 0.743, yielding an error of $1.779 \times 10^{-3}$ at $M = 12,288$ in 10.809 seconds.

Table 4.19: Mesh refinement study of the Gemini Pro Direct Collocation Constant BEM solver on the Perturbed Sphere domain.

| **Elements ($N_e$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---|---|---|---|---|
| 768 | 4.622e-03 | 0.714 | 0.157 | 0.002 | 0.873 |
| 3,072 | 3.638e-03 | 4.369 | 0.356 | 0.008 | 4.732 |
| 12,288 | 1.654e-03 | 13.869 | 6.327 | 0.023 | 20.219 |

Table 4.20: Mesh refinement study of the ChatGPT Direct Collocation Constant BEM solver on the Perturbed Sphere domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---|---|---|---|---|
| 768 | 4.979e-03 | 0.064 | 0.036 | 0.001 | 0.101 |
| 3,072 | 3.923e-03 | 0.580 | 0.315 | 0.004 | 0.899 |
| 12,288 | 1.779e-03 | 5.258 | 5.528 | 0.023 | 10.809 |

**Scheme 2: Direct Formulation — Collocation — Linear Elements**

**Generative Efficiency:** Gemini Pro generated a functional solver in the first attempt ($I_c = 0$). ChatGPT failed to produce a converging solver even after 5 iterations ($I_c = 5$), rendering its numerical outputs non-functional.

**Computational Performance:** Both models failed to achieve the theoretical $O(h^2)$ convergence rate expected for continuous linear elements. Gemini Pro recorded a

convergence order of 0.841, reaching a relative $L_2$ error of $1.386 \times 10^{-2}$ at $M = 12,288$ in 5.634 seconds. ChatGPT exhibited massive instability with a convergence slope of 0.228, yielding a terminal relative $L_2$ error of $4.545 \times 10^{-1}$ at $M = 12,288$ in 127.116 seconds.

Table 4.21: Mesh refinement study of the Gemini Pro Direct Collocation Linear BEM solver on the Perturbed Sphere domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---|---|---|---|---|
| 768 | 4.446e-02 | 0.017 | 0.014 | 0.002 | 0.032 |
| 3,072 | 3.277e-02 | 0.119 | 0.115 | 0.005 | 0.238 |
| 12,288 | 1.386e-02 | 1.300 | 4.309 | 0.025 | 5.634 |

Table 4.22: Mesh refinement study of the ChatGPT Direct Collocation Linear BEM solver on the Perturbed Sphere domain.

| **Elements ($M$)** | **Rel $L_2$ Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---|---|---|---|---|
| 768 | 6.233e-01 | 0.033 | 0.036 | 0.023 | 0.091 |
| 3,072 | 8.622e-01 | 0.275 | 1.637 | 0.048 | 1.959 |
| 12,288 | 4.545e-01 | 2.752 | 124.213 | 0.151 | 127.116 |

**Scheme 3: Direct Formulation — Galerkin — Linear Elements**

**Generative Efficiency:** Both models required four attempts ($I_c = 3$) to generate a stable solver. Gemini Pro employed rigorous 4D Sauter-Schwab coordinate transformations for singular integrals and near-field interactions. ChatGPT relied on a standard 144-point mapped Gauss-Legendre Duffy transformation for its singular evaluations.

**Computational Performance:** Both models failed to attain the theoretical $O(h^2)$ convergence rate expected for continuous linear elements on the highly perturbed geometry. Gemini Pro achieved an observed convergence order of 0.978, reaching a relative $L_2$ error of $1.158 \times 10^{-2}$ at $M = 12,288$ in 84.667 seconds. ChatGPT recorded a lower convergence order of 0.772, yielding a relative $L_2$ error of $1.068 \times 10^{-2}$ at $M = 12,288$ in 38.765 seconds.

Table 4.23: Mesh refinement study of the Gemini Pro Direct Galerkin Linear BEM solver on the Perturbed Sphere domain.

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---|---|---|---|---|
| 768 | 4.496e-02 | 0.433 | 0.008 | 0.001 | 0.442 |
| 3,072 | 2.960e-02 | 3.681 | 0.119 | 0.003 | 3.802 |
| 12,288 | 1.158e-02 | 77.898 | 6.757 | 0.009 | 84.667 |

Table 4.24: Mesh refinement study of the ChatGPT Direct Galerkin Linear BEM solver on the Perturbed Sphere domain.

| **Elements** ($M$) | **Rel** $L_2$ **Error** | **Setup (s)** | **Solve (s)** | **Eval (s)** | **Total (s)** |
|---:|---|---|---|---|---|
| 768 | 3.113e-02 | 3.798 | 0.005 | 2.788 | 6.592 |
| 3,072 | 2.702e-02 | 2.100 | 0.130 | 0.008 | 2.238 |
| 12,288 | 1.068e-02 | 28.943 | 9.774 | 0.045 | 38.765 |

[a] The setup time for the initial mesh includes Numba JIT compilation overhead.

### Visual Summary

Figure 4.6 provides a visual summary of the performance for the finest meshes evaluated across all formulations in the perturbed sphere domain, illustrating the time-to-accuracy trade-offs and capability limits discussed in the preceding sections. The plot is on a log-log scale.

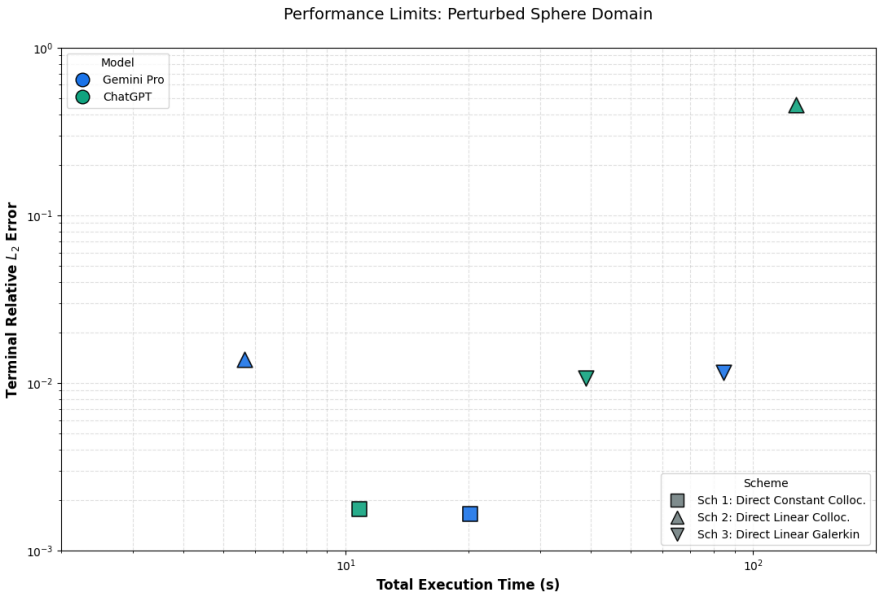



Figure 4.6: $L_2$ error vs. total execution time for all evaluated schemes at the finest mesh resolutions for the perturbed sphere domain

# CHAPTER 5

# CONCLUSIONS AND FUTURE WORK

## 5.1 SUMMARY OF FINDINGS

This dissertation explored the practical utility of Large Language Models (LLMs)—specifically Gemini Pro, ChatGPT (Thinking), and Claude Sonnet (for 2D domains)—as assistive tools for generating Boundary Element Method (BEM) solvers for Laplace's equation. Rather than establishing definitive boundaries for LLM behavior, this study presents observational trends, acknowledging the inherently fluid nature of generative AI. Overall, the findings indicate that state-of-the-art models perform well enough to serve as valuable companions, generally achieving engineering-level accuracy for the tested domains when appropriately guided.

**The Importance of Detailed Prompting:**

The successful translation of mathematical concepts from English into functional Python code relies heavily on the specificity of the initial instructions. Detailed, highly structured prompting was routinely observed to bridge the gap between theoretical problem definitions and executable code. By explicitly defining conventions—such as outward normal orientations and specific integral treatments—engineers can effectively guide the LLM, reducing the likelihood of assumption-based algorithmic errors.

**Probabilistic Outputs and Stochastic Reliability:**

A fundamental limitation of evaluating LLMs in computational mechanics is their non-deterministic nature. Because these models function as probabilistic text generators rather than deterministic compilers, their outputs exhibit inherent variance. An identical prompt may yield a bug-free, perfectly stable solver on one day, while on another day, it might require multiple iterations to correct minor sign errors or properly handle singular

limits. Consequently, LLMs are best viewed as capable but occasionally unpredictable copilots, where zero-shot success is possible but never guaranteed.

**Temporal Variability and Model Evolution:**

Furthermore, the rapid pace of AI development introduces a temporal inconsistency to any benchmark. Foundation models undergo continuous updates, safety fine-tuning, and silent parameter adjustments. A specific test case that caused a model to fail during the early stages of this research might be solved flawlessly by the current version of that same model and prompt. Therefore, the specific successes and failures documented in this work represent a specific snapshot in time, rather than a permanent ceiling on LLM capabilities.

## 5.2 CONCLUDING REMARKS

In conclusion, LLMs are highly effective tools for accelerating BEM code development, despite their inherent unpredictability. Their capacity to rapidly draft, refine, and debug Python implementations makes them valuable assets for computational engineers. Given their probabilistic nature and continuous evolution, these models are best utilized as advanced assistants rather than independent problem solvers. Human oversight remains essential to verify mathematical accuracy, ensure physical consistency, and direct the overall numerical strategy.

## 5.3 DIRECTIONS FOR FUTURE WORK

While this dissertation establishes a baseline for LLM-assisted code generation within standard Boundary Element Methods, the rapid evolution of both artificial intelligence and computational mechanics presents numerous avenues for future research. The immediate logical progression is to evaluate the capacity of foundation models to synthesize algorithms from recent, state-of-the-art computational physics literature, moving beyond established canonical frameworks.

Specific areas of future investigation include:

**1. Integration of Neural Methods:**

Recent advancements have seen the blending of traditional boundary integrals with machine learning architectures. Future work could evaluate the ability of LLMs to implement hybrid formulations, such as the Boundary Neuron Method (BNM) or Random Feature (RF) approximations, where neural networks are utilized to parameterize unknown boundary functions or accelerate integral evaluations [7].

**2. Advanced Fast Solvers:**

A critical next step is to test the generative capabilities of LLMs in implementing optimal complexity algorithms. This includes evaluating whether an LLM can correctly synthesize the hierarchical octree logic and multipole expansion translations required for the Fast Multipole Method (FMM) [8], or formulate advanced Asymmetric Preconditioning techniques to accelerate iterative Krylov subspace solvers [9].

**3. Stochastic and Time-Domain Formulations:**

Expanding the temporal and probabilistic scope of the solvers represents another crucial testing ground. Future research should assess the ability of LLMs to generate Monte Carlo-based "Walk-on-Boundary" algorithms for high-dimensional Laplace or Poisson problems [10]. Additionally, testing the implementation of Generalized Convolution Quadrature (CQ) would evaluate the models' capacity to handle rigorous time-stepping formulations for transient wave or diffusion phenomena.

**4. Multi-Agent Systems and Autonomous Debugging:**

Future studies should investigate the use of Multi-Agent LLM architectures—where a "Coder" agent writes the BEM implementation while a secondary "Reviewer" or "Physicist" agent autonomously evaluates the mathematics and matrix conditioning. Because 3D BEM simulations are inherently computationally expensive, relying purely on execution-based testing to generate feedback is highly inefficient. An autonomous reviewer

agent could mitigate this by performing rigorous static analysis—verifying integral formulations, jump coefficients, and normal orientations—before the code is ever executed. Automating this pre-computation feedback loop would enable rapid, iterative debugging without incurring the prohibitive costs of running the solver, potentially eliminating the current need for human intervention in resolving geometric or mathematical nuances.

# REFERENCES


[1] George Green. *An Essay on the Application of Mathematical Analysis to the Theories of Electricity and Magnetism*. Printed for the author by T. Wheelhouse, Nottingham, 1828.

[2] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, volume 30, 2017.

[3] Frank J Rizzo. An integral equation approach to boundary value problems of classical elastostatics. *Quarterly of Applied Mathematics*, 25(1):83–95, 1967.

[4] Viktor Dmitrievich Kupradze. *Potential Methods in the Theory of Elasticity*. Israel Program for Scientific Translations, Jerusalem, 1965.

[5] Rainer Kress. *Linear Integral Equations*. Springer-Verlag, Berlin Heidelberg, 1989.

[6] Youcef Saad and Martin H Schultz. GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM Journal on Scientific and Statistical Computing*, 7(3):856–869, 1986.

[7] Ye Lin, Wentao Liu, Young Ju Lee, and Jiwei Jia. Boundary neuron method for solving partial differential equations. *arXiv preprint arXiv:2603.26643*, 2026.

[8] Vladimir Rokhlin. Rapid solution of integral equations of classical potential theory. *Journal of Computational Physics*, 60(2):187–207, 1985.

[9] Jiong Chen, Florian T Schäfer, and Mathieu Desbrun. Lightning-fast boundary element method. *ACM Transactions on Graphics (TOG)*, 44(4), 2025.

[10] Rohan Sawhney, Bailey Miller, Ioannis Gkioulekas, and Keenan Crane. Walk on stars: A grid-free monte carlo method for pdes with neumann boundary conditions. *ACM Transactions on Graphics (TOG)*, 42(4):1–19, 2023.

# CURRICULUM VITAE

**NAME** Pranav

**DATE OF BIRTH** 22 June 2004

**EDUCATION QUALIFICATIONS**

**2026 BACHELORS OF TECHNOLOGY**

| | |
|---|---|
| Institution | Indian Institute of Technology, Madras |
| Specialization | Mechanical Engineering |

**INTERDISCIPLINARY DUAL DEGREE**

| | |
|---|---|
| Institution | Indian Institute of technology, Madras |
| Specialization | Data Science |
| Registration Date | 30 November 2021 |